PRACTICAL 1: Creation of Test Suite Using Selenium IDE

Aim: To install Selenium IDE and create a test suite containing minimum four test cases to validate different web page formats such as HTML page, XML response, JSON API output, and dynamic form page. Execute the test suite and export the automation script in WebDriver format.

Requirements:

Mozilla Firefox (Recommended over Chrome due to extension availability)
Selenium IDE extension
VS Code (or Notepad)
Python (for local web server)
Internet connection

Step 1: Create the Local Testing Files

Create a folder named Practical_1_SeleniumIDE. Inside it, create the following files using VS Code or Notepad:

File 1: index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>STQA Practical 1 - HTML</title>
</head>
<body>
    <h1>Software Testing and Quality Assurance</h1>
    <p>This is the HTML page used for Selenium IDE testing.</p>
    <p>Practical 1 - HTML Page Validation</p>
</body>
</html>
```

File 2: xml_response.html (Note: We wrap XML in HTML so Firefox allows Selenium to read it)

```
<!DOCTYPE html>
<html>
<head>
    <title>XML Response Simulation</title>
</head>
<body>
    <h3>XML Data Received:</h3>
    <p>Student Name: <span id="studentName">Vicky</span></p>
    <pre>
&lt;?xml version="1.0" encoding="UTF-8"?&gt;
&lt;student&gt;
    &lt;name&gt;Vicky&lt;/name&gt;
    &lt;course&gt;B.Sc. Computer Science&lt;/course&gt;
    &lt;subject&gt;STQA&lt;/subject&gt;
    &lt;marks&gt;78&lt;/marks&gt;
&lt;/student&gt;
    </pre>
</body>
</html>
```

File 3: json_response.html (Same reasoning as XML, we wrap JSON in HTML)

```
<!DOCTYPE html>
<html>
<head>
    <title>JSON API Response Simulation</title>
</head>
<body>
    <h3>JSON Data Received from API:</h3>
    <p>User Name: <span id="userName">Leanne Graham</span></p>
    <h4>Raw JSON Response:</h4>
    <pre>
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz"
}
    </pre>
</body>
</html>
```

File 4: dynamic.html

```
<!DOCTYPE html>
<html>
<head>
    <title>STQA Practical 1 - Dynamic Form</title>
    <script>
        function submitForm() {
            var name = document.getElementById("name").value;
            var course = document.getElementById("course").value;
            if (name.trim() === "") {
                document.getElementById("message").innerText = "Please enter your name";
                return;
            }
            document.getElementById("message").innerText = "Form submitted successfully - " +
name + " - " + course;
        }
    </script>
</head>
<body>
    <h2>Student Registration Form</h2>
    <label>Name:</label>
    <input type="text" id="name"><br><br>
    <label>Course:</label>
    <select id="course">
        <option value="Computer Science">Computer Science</option>
```

```
        <option value="Information Technology">Information Technology</option>
        <option value="Commerce">Commerce</option>
    </select><br><br>
    <label><input type="checkbox" id="agree"> I agree</label><br><br>
    <button id="submit" onclick="submitForm()">Submit</button>
    <p id="message"></p>
</body>
</html>
```

Step 3: Install Selenium IDE on Firefox

TC01_HTML_Page

Command	Target	Value
open	http://localhost:8000/index.html	
assert text	css=h1	Software Testing and Quality Assurance

TC02_XML_Response

Command	Target	Value
open	http://localhost:8000/xml_response.html	
assert text	css=#studentName	Vicky

TC03_JSON_API

Command	Target	Value
open	http://localhost:8000/json_response.html	
assert text	css=#userName	Leanne Graham

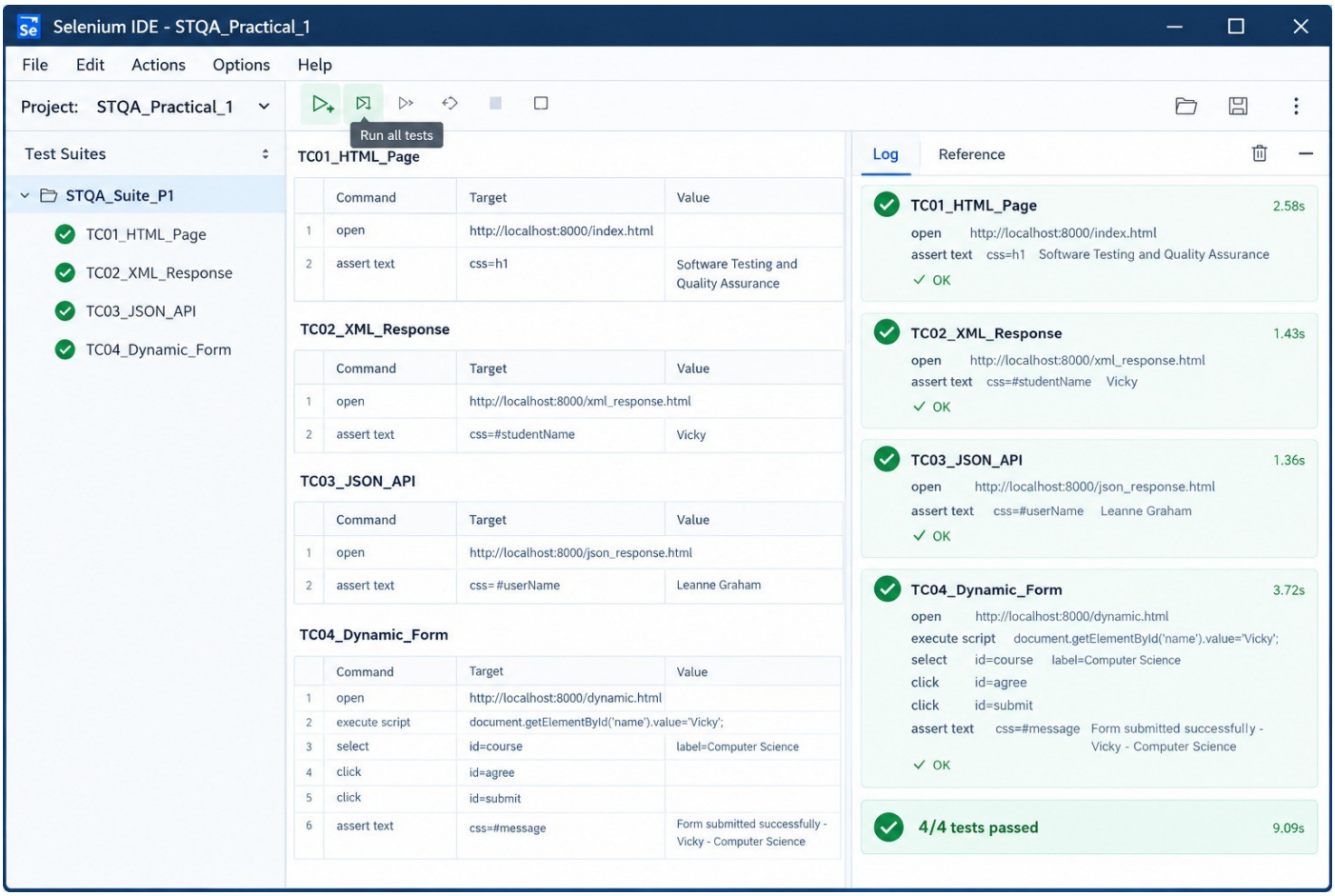
TC04_Dynamic_Form

Command	Target	Value
open	http://localhost:8000/dynamic.html	
execute script	document.getElementById('name').value='Vicky';	
select	id=course	label=Computer Science
click	id=agree	

Command	Target	Value
click	id=submit	
assert text	css=#message	Form submitted successfully - Vicky - Computer Science

Step 5: Run the Suite and Export

Result:
"All four test cases (HTML, XML, JSON, Dynamic Form) were created and executed successfully in Selenium IDE. The test suite was exported to WebDriver format."



PRACTICAL 2: Cross-Website Functional Testing

Aim: To develop and execute a Selenium IDE test suite for two different websites and verify functional operations such as clicking links, filling forms, selecting dropdown options, and verifying displayed content.

Websites Tested:

1. SauceDemo (https://www.saucedemo.com/)
2. The Internet (https://the-internet.herokuapp.com/)

Step 1: Create the Project

Step 2: Create Test Cases (Debugged Commands)

Because SauceDemo is built on React, standard type and sendKeys commands fail on Firefox. We must use execute script with specific event dispatchers.

TC01_SauceDemo_Login

Command	Target	Value
open	https://www.saucedemo.com/	
execute script	var el = document.getElementById('user-name'); var setter = Object.getOwnPropertyDescriptor(window.HTMLInputElement.prototype, 'value').set; setter.call(el, 'standard_user'); el.dispatchEvent(new Event('input', { bubbles: true }));	
execute script	var el = document.getElementById('password'); var setter = Object.getOwnPropertyDescriptor(window.HTMLInputElement.prototype, 'value').set; setter.call(el, 'secret_sauce'); el.dispatchEvent(new Event('input', { bubbles: true }));	
click	id=login-button	
assert text	css=.title	Products

TC02_SauceDemo_Cart

Command	Target	Value
open	https://www.saucedemo.com/	
execute script	*(Paste the same Username script from TC01)*	
execute script	*(Paste the same Password script from TC01)*	
click	id=login-button	

Command	Target	Value
execute script	var button = null; var attempts = 0; while (button === null && attempts < 20) { button = document.querySelector('button[id*="add-to-cart-sauce-labs-backpack"]'); if (button === null) { attempts++; } } if (button !== null) { button.click(); }	
pause	1000	
click	css=.shopping_cart_link	
assert text	css=.inventory_item_name	Sauce Labs Backpack

TC03_TheInternet_Dropdown

Command	Target	Value
open	https://the-internet.herokuapp.com/dropdown	
select	id=dropdown	label=Option 2
assert text	css=h3	Dropdown List

TC04_TheInternet_DynamicControl

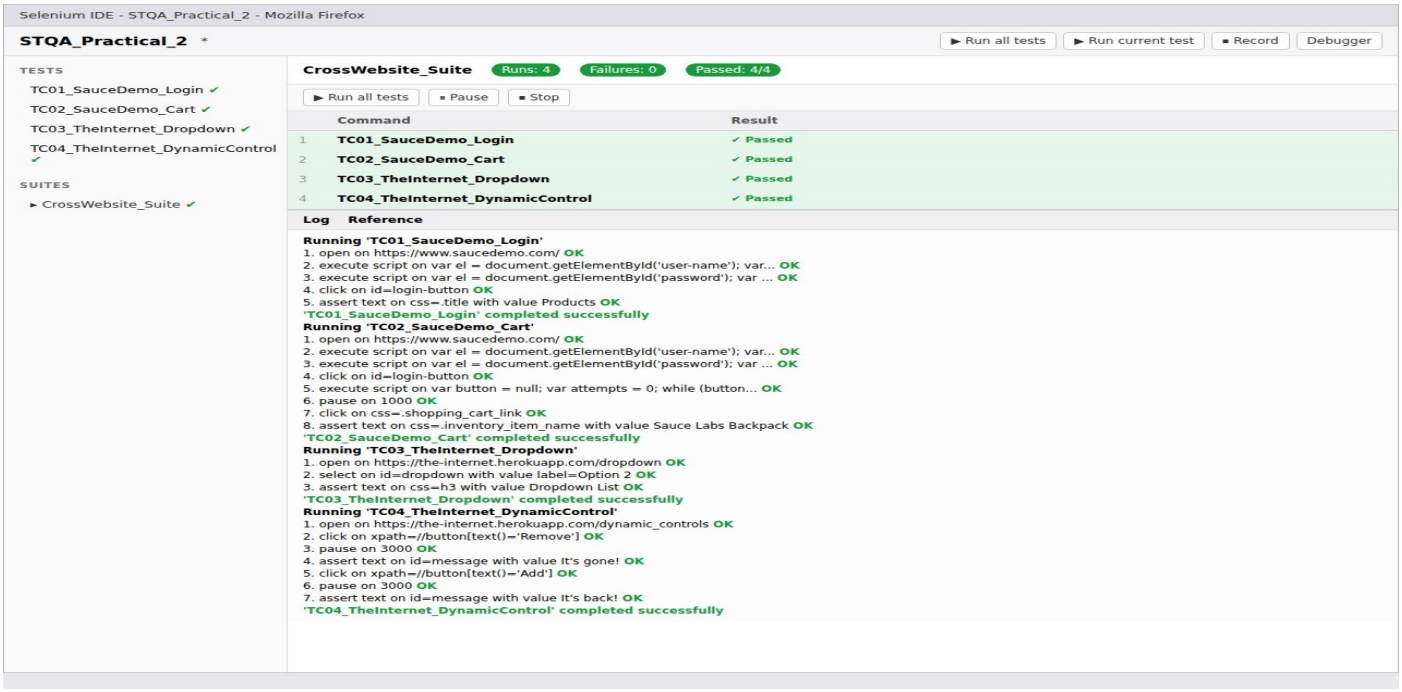
Command	Target	Value
open	https://the-internet.herokuapp.com/dynamic_controls	
click	xpath=//button[text()='Remove']	
pause	3000	
assert text	id=message	It's gone!
click	xpath=//button[text()='Add']	
pause	3000	
assert text	id=message	It's back!

Step 3: Run the Suite and Export

Result:

"The test suite was successfully created and executed on two different websites (SauceDemo and The Internet). Functional operations such as login, adding to cart, selecting a dropdown option, and handling dynamic elements were tested. Execution logs were generated and analyzed."

Stqa 2nd lecture : 2,4,7,10,14,15,20,22,23,30,34,36,40,42,44,45



Practical 3: Manual Test Case Design and Execution — Complete Step-by-Step

Aim: To prepare a Test Plan, Test Scenarios, and Test Cases for a given web application, execute the test cases manually, and prepare a Test Execution Report and Defect Report as per the STLC process.

Requirements:

- Microsoft Excel (or Google Sheets, or Notepad for CSV editing)
- Google Chrome browser
- SauceDemo access: https://www.saucedemo.com/

Theory:

Manual testing is the process of testing software without using any automation tools. The tester plays the role of an end user and checks whether the application behaves as expected. The Software Testing Life Cycle (STLC) is the systematic process followed in testing, which includes requirement analysis, test planning, test case design, environment setup, test execution, defect reporting, and test closure.

Procedure

Step 1: Create the Folder and Files

Step 2: Write the Test Plan

Open 1_TestPlan.csv in Notepad and type:

```
Field,Details
Project Name,SauceDemo Manual Testing
Application,SauceDemo Web Application
URL,https://www.saucedemo.com/
Testing Type,Functional Testing
Testing Method,Manual Testing
Browser,Google Chrome
Objective,"To verify the main functional modules of the SauceDemo web application and identify any defects"
Test Data,"Valid credentials (standard_user/secret_sauce) and invalid credentials"
Scope,"Login, Products, Cart, Checkout, Logout"
Out of Scope,"Database testing, Server-side testing, Payment gateway, Performance testing"
Entry Criteria,"Application is accessible and test data is available"
Exit Criteria,"All planned test cases are executed and results are documented"
Deliverables,"Test Scenarios, Test Cases, Test Execution Report, Defect Report"
Prepared By,[Your Name]
Roll Number,[Your Roll Number]
Date,[Today's Date]
```

Step 3: Write the Test Scenarios

Open 2_TestScenarios.csv in Notepad and type:

```
Scenario ID,Module,Test Scenario
TS01,Login,Verify login with valid credentials
TS02,Login,Verify login with invalid credentials
TS03,Login,Verify validation when login fields are empty
TS04,Products,Verify that all products are displayed with correct details
```

TS05,Products,Verify that a product can be added to the cart
TS06,Cart,Verify that the added product appears in the cart
TS07,Cart,Verify that a product can be removed from the cart
TS08,Checkout,Verify that the checkout information form is displayed
TS09,Products,Verify that the sorting dropdown works correctly
TS10,Logout,Verify that the user can log out successfully
Save the file.

Step 4: Write the Test Cases

Open 3_TestCases.csv in Notepad. Copy the following. Note that we've used semicolons (;) inside the test steps so the CSV doesn't break. The last three columns (Actual Result, Status, Remarks) are empty for now — we'll fill them after execution.

TC ID,Scenario ID,Test Case,Pre-Condition,Test Steps,Test Data,Expected Result,Actual Result,Status,Remarks
TC01,TS01,Verify login with valid credentials,Browser is open and SauceDemo login page is accessible,1. Open https://www.saucedemo.com/ ; 2. Enter standard_user in Username ; 3. Enter secret_sauce in Password ; 4. Click Login button,Username: standard_user ; Password: secret_sauce,User should be redirected to the Products page,,,
TC02,TS02,Verify login with invalid credentials,Browser is open,1. Open https://www.saucedemo.com/ ; 2. Enter wrong_user in Username ; 3. Enter wrong_pass in Password ; 4. Click Login button,Username: wrong_user ; Password: wrong_pass,Error message should be displayed and user should remain on the login page,,,
TC03,TS03,Verify validation when login fields are empty,Browser is open,1. Open https://www.saucedemo.com/ ; 2. Leave both Username and Password empty ; 3. Click Login button,(Empty fields),Error message Username is required should be displayed,,,
TC04,TS04,Verify that all products are displayed with correct details,User is logged in,1. Login with valid credentials ; 2. Observe the Products page ; 3. Check product names prices and images,NA,All 6 products should be displayed with names prices and images,,,
TC05,TS05,Verify that a product can be added to the cart,User is logged in,1. Login with valid credentials ; 2. Click Add to Cart on Sauce Labs Backpack ; 3. Observe the cart badge count,Product: Sauce Labs Backpack,Cart badge count should increase to 1,,,
TC06,TS06,Verify that the added product appears in the cart,Product is added to cart,1. Click the shopping cart icon ; 2. Observe the product listed in the cart,Product: Sauce Labs Backpack,Sauce Labs Backpack should be displayed in the cart with price $29.99,,,
TC07,TS07,Verify that a product can be removed from the cart,Product is added to cart,1. Open the cart ; 2. Click Remove button for the product,Product: Sauce Labs Backpack,Product should be removed and the cart badge should disappear,,,
TC08,TS08,Verify that the checkout information form is displayed,Product is added to cart,1. Open the cart ; 2. Click Checkout button ; 3. Observe the form,NA,First Name Last Name and Zip or Postal Code fields should be displayed,,,
TC09,TS09,Verify that the sorting dropdown works correctly,User is logged in,1. Login with valid credentials ; 2. Click the sorting dropdown ; 3. Select Price low to high ; 4. Observe the product order,Sort Option: Price (low to high),Products should be rearranged from lowest price to highest price,,,
TC10,TS10,Verify that the user can log out successfully,User is logged in,1. Click the hamburger menu icon ; 2. Click Logout,NA,User should be logged out and returned to the login page,,,

Step 5: Execute the Test Cases Manually

Open Google Chrome and go to **https://www.saucedemo.com/**. Keep this tab open. Also keep **3_TestCases.csv** open in Notepad on the side. we'll fill in the results as we go.

TC01 — Valid Login

In Chrome:

1. Type standard_user in Username.
2. Type secret_sauce in Password.
3. Click **Login**.

What we see: The Products page opens with 6 products (Backpack, Bike Light, Bolt T-Shirt, Fleece Jacket, Onesie, T-Shirt Red) with names, prices, and images.

TC02 — Invalid Login

In Chrome:

What you see: A red error box: Epic sadface: Username and password do not match any user in this service

TC03 — Empty Login Fields

In Chrome:

What you see: A red error box: Epic sadface: Username is required

.

TC04 — Products Display

In Chrome:

What you see: 6 products displayed with names, prices, and images.

TC05 — Add to Cart

In Chrome:

What you see: The button changes to "Remove." A red badge with **1** appears on the cart icon at the top-right.

TC06 — Verify Product in Cart

In Chrome:

What you see: The cart page shows Sauce Labs Backpack with price **$29.99** and quantity 1.

TC07 — Remove Product

In Chrome:

What you see: The product disappears. The cart badge disappears too.

TC08 — Checkout Form

In Chrome:

What you see: A form with three fields — First Name, Last Name, Zip/Postal Code.

TC09 — Product Sorting

In Chrome:

What you see: Products rearrange. The first product is now **Sauce Labs Onesie ($7.99)** and the last is **Sauce Labs Fleece Jacket ($49.99)**.

TC10 — Logout

In Chrome:

What you see: You're back on the login page with the Swag Labs logo and the list of accepted usernames.

Step 6: Update the Test Execution Report

Open 4_TestExecutionReport.csv in Notepad. Fill in the Actual Result, Status, and Defect ID for each row. Here's what it should look like:

TC ID,Test Case,Expected Result,Actual Result,Status,Defect ID
TC01,Valid Login,Products page opens,Products page displayed with 6 items,Pass,--
TC02,Invalid Login,Error message shown,Error message "Epic sadface: Username and password do not match" displayed,Pass,--
TC03,Empty Login Fields,Validation message shown,Error message "Epic sadface: Username is required" displayed,Pass,--
TC04,Products Display,All products displayed,All 6 products displayed with names and prices,Pass,--
TC05,Add to Cart,Product added to cart,Cart badge count increased to 1,Pass,--
TC06,Verify Product in Cart,Product shown in cart,Sauce Labs Backpack displayed with price $29.99,Pass,--
TC07,Remove Product,Product removed,Product removed and cart badge disappeared,Pass,--
TC08,Checkout Form,Checkout form displayed,First Name Last Name and Zip fields displayed,Pass,--
TC09,Product Sorting,Products sorted,Products rearranged from $7.99 to $49.99,Pass,--
TC10,Logout,User logged out,Redirected to login page,Pass,--
Summary,,,,,
Total Test Cases,10,,,,
Passed,10,,,,
Failed,0,,,,
Blocked,0,,,,
Pass Percentage,100%,,,,
Save the file.

Step 7: The Defect Report
Open 5_DefectReport.csv in Notepad. Since all tests passed, write:

Defect ID,TC ID,Module,Defect Title,Steps to Reproduce,Expected Result,Actual Result,Severity,Priority,Status,Remarks
N/A,N/A,N/A,No defect observed during execution,N/A,N/A,N/A,N/A,N/A,Closed,"All 10 test cases passed successfully. No deviations from expected behavior were observed."
Save the file.

Step 8: Open Everything in Excel and Take Screenshots

1_TestPlan.csv - Excel

File Home Insert Page Layout Formulas Data Review View

A1 fx Field

	A	B
1	Field	Details
2	Project Name	SauceDemo Manual Testing
3	Application	SauceDemo Web Application
4	URL	https://www.saucedemo.com/
5	Testing Type	Functional Testing
6	Testing Method	Manual Testing
7	Browser	Google Chrome
8	Objective	To verify the main functional modules of the SauceDemo web application and identify any defects
9	Test Data	Valid credentials (standard_user/secret_sauce) and invalid credentials
10	Scope	Login, Products, Cart, Checkout, Logout
11	Out of Scope	Database testing, Server-side testing, Payment gateway, Performance testing
12	Entry Criteria	Application is accessible and test data is available
13	Exit Criteria	All planned test cases are executed and results are documented
14	Deliverables	Test Scenarios, Test Cases, Test Execution Report, Defect Report
15	Prepared By	Neha Lodh
16	Roll Number	[Your Roll Number]
17	Date	13/10/2026

1_TestPlan +

Ready

2_TestScenarios.csv - Excel

File Home Insert Page Layout Formulas Data Review View

A1 fx Scenario ID

	A	B	C
1	Scenario ID	Module	Test Scenario
2	TS01	Login	Verify login with valid credentials
3	TS02	Login	Verify login with invalid credentials
4	TS03	Login	Verify validation when login fields are empty
5	TS04	Products	Verify that all products are displayed with correct details
6	TS05	Products	Verify that a product can be added to the cart
7	TS06	Cart	Verify that the added product appears in the cart
8	TS07	Cart	Verify that a product can be removed from the cart
9	TS08	Checkout	Verify that the checkout information form is displayed
10	TS09	Products	Verify that the sorting dropdown works correctly
11	TS10	Logout	Verify that the user can log out successfully

2_TestScenarios +

Ready

3_TestCases.csv - Excel

File Home Insert Page Layout Formulas Data Review View

A1 fx TC ID

	A	B	C	D	E	F	G	H	I	J
1	TC ID	Scenario ID	Test Case	Pre-Condition	Test Steps	Test Data	Expected Result	Actual Result	Status	Remarks
2	TC01	TS01	Verify login with valid credentials	Browser is open and SauceDemo login page is accessible	1. Open https://www.saucedemo.com/ ; 2. Enter standard_user in Username ; 3. Enter secret_sauce in Password ; 4. Click Login button	Username: standard_user ; Password: secret_sauce	User should be redirected to the Products page	Products page displayed with 6 items	Pass	
3	TC02	TS02	Verify login with invalid credentials	Browser is open	1. Open https://www.saucedemo.com/ ; 2. Enter wrong_user in Username ; 3. Enter wrong_pass in Password ; 4. Click Login button	Username: wrong_user ; Password: wrong_pass	Error message should be displayed and user should remain on the login page	Error message "Epic sadface: Username and password do not match any user in this service" displayed	Pass	
4	TC03	TS03	Verify validation when login fields are empty	Browser is open	1. Open https://www.saucedemo.com/ ; 2. Leave both Username and Password empty ; 3. Click Login button	(Empty fields)	Error message Username is required should be displayed	Error message "Epic sadface: Username is required" displayed	Pass	
5	TC04	TS04	Verify that all products are displayed with correct details	User is logged in	1. Login with valid credentials ; 2. Observe the Products page ; 3. Check product names prices and images	NA	All 6 products should be displayed with names prices and images	All 6 products displayed with names prices and images	Pass	
6	TC05	TS05	Verify that a product can be added to the cart	User is logged in	1. Login with valid credentials ; 2. Click Add to Cart on Sauce Labs Backpack ; 3. Observe the cart badge count	Product: Sauce Labs Backpack	Cart badge count should increase to 1	Cart badge count increased to 1	Pass	
7	TC06	TS06	Verify that the added product appears in the cart	Product is added to cart	1. Click the shopping cart icon ; 2. Observe the product listed in the cart	Product: Sauce Labs Backpack	Sauce Labs Backpack should be displayed in the cart with price $29.99	Sauce Labs Backpack displayed in cart with price $29.99	Pass	
8	TC07	TS07	Verify that a product can be removed from the cart	Product is added to cart	1. Open the cart ; 2. Click Remove button for the product	Product: Sauce Labs Backpack	Product should be removed and the cart badge should disappear	Product removed and cart badge disappeared	Pass	
9	TC08	TS08	Verify that the checkout information form is displayed	Product is added to cart	1. Open the cart ; 2. Click Checkout button ; 3. Observe the form	NA	First Name Last Name and Zip or Postal Code fields should be displayed	First Name Last Name and Zip or Postal Code fields displayed	Pass	
10	TC09	TS09	Verify that the sorting dropdown works correctly	User is logged in	1. Login with valid credentials ; 2. Click the sorting dropdown ; 3. Select Price low to high ; 4. Observe the product order	Sort Option: Price (low to high)	Products should be rearranged from lowest price to highest price	Products rearranged from $7.99 to $49.99	Pass	
11	TC10	TS10	Verify that the user can log out successfully	User is logged in	1. Click the hamburger menu icon ; 2. Click Logout	NA	User should be logged out and returned to the login page	User logged out and redirected to login page	Pass	

3_TestCases +

Ready

4_TestExecutionReport.csv - Excel

File Home Insert Page Layout Formulas Data Review View

A1 fx TC ID

	A	B	C	D	E	F
1	TC ID	Test Case	Expected Result	Actual Result	Status	Defect ID
2	TC01	Valid Login	Products page opens	Products page displayed with 6 items	Pass	--
3	TC02	Invalid Login	Error message shown	Error message "Epic sadface: Username and password do not match" displayed	Pass	--
4	TC03	Empty Login Fields	Validation message shown	Error message "Epic sadface: Username is required" displayed	Pass	--
5	TC04	Products Display	All products displayed	All 6 products displayed with names and prices	Pass	--
6	TC05	Add to Cart	Product added to cart	Cart badge count increased to 1	Pass	--
7	TC06	Verify Product in Cart	Product shown in cart	Sauce Labs Backpack displayed with price $29.99	Pass	--
8	TC07	Remove Product	Product removed	Product removed and cart badge disappeared	Pass	--
9	TC08	Checkout Form	Checkout form displayed	First Name Last Name and Zip fields displayed	Pass	--
10	TC09	Product Sorting	Products sorted	Products rearranged from $7.99 to $49.99	Pass	--
11	TC10	Logout	User logged out	Redirected to login page	Pass	--
12	Summary					
13	Total Test Cases	10				
14	Passed	10				
15	Failed	0				
16	Blocked	0				
17	Pass Percentage	100%				

4_TestExecutionReport +

Ready

5_DefectReport.csv - Excel

File Home Insert Page Layout Formulas Data Review View

A1 fx Defect ID

	A	B	C	D	E	F	G	H	I	J	K
1	Defect ID	TC ID	Module	Defect Title	Steps to Reproduce	Expected Result	Actual Result	Severity	Priority	Status	Remarks
2	N/A	N/A	N/A	No defect observed during execution	N/A	N/A	N/A	N/A	N/A	Closed	All 10 test cases passed successfully. No deviations from expected behavior were observed.

5_DefectReport +

Ready

PRACTICAL 4: Selenium Server (Grid) Configuration and Remote Execution

Aim

To install and configure Selenium Server in Hub-Node architecture, execute a test script on multiple browsers using RemoteWebDriver, and compare cross-browser test results.

Requirements:

JDK 17 (we used C:\Program Files\Java\jdk-17)
Eclipse IDE (2025-06 or later)
Selenium Server 4.49.0 jar
Selenium Java Client 4.49.0 (via Maven)
Google Chrome and Mozilla Firefox installed
Internet connection

PART 1: Install JDK 17

PART 2: Verify Java 17 Without Changing System PATH

PART 3: Download Selenium Server

PART 4: Start the Hub

1. Open Command Prompt.
2. Navigate to the folder:

cd "C:\Users\Abr\OneDrive\Desktop\STQA_Practicals TYCS\Practical_4_SeleniumGrid"
(Adjust the path if yours is different.)

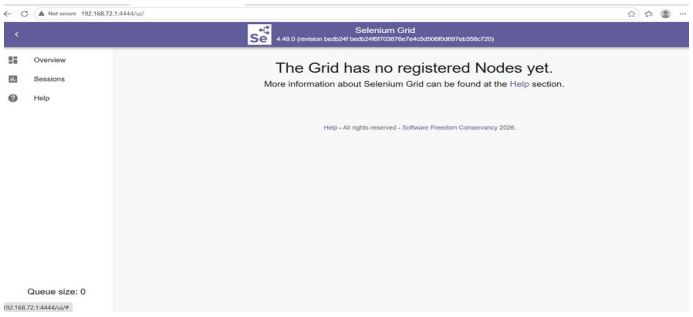
3. Start the Hub using the full path to JDK 17's java.exe:

"C:\Program Files\Java\jdk-17\bin\java.exe" -jar selenium-server-4.49.0.jar hub

4. You will see a lot of log output. After about 20 seconds, the important line appears:

Started Selenium Hub 4.49.0 (revision ...): http://192.168.72.1:4444





PART 5: Verify the Hub in Browser

1. Open Firefox or Chrome.
2. Go to: http://localhost:4444/ui/

PART 6: Start the Node

1. Open a **second** Command Prompt (Windows key + R → cmd → Enter).
2. Navigate to the same folder:
3. cd "C:\Users\Abr\OneDrive\Desktop\STQA_Practicals TYCS\Practical_4_SeleniumGrid"

"C:\Program Files\Java\jdk-17\bin\java.exe" -jar selenium-server-4.49.0.jar node --selenium-manager true

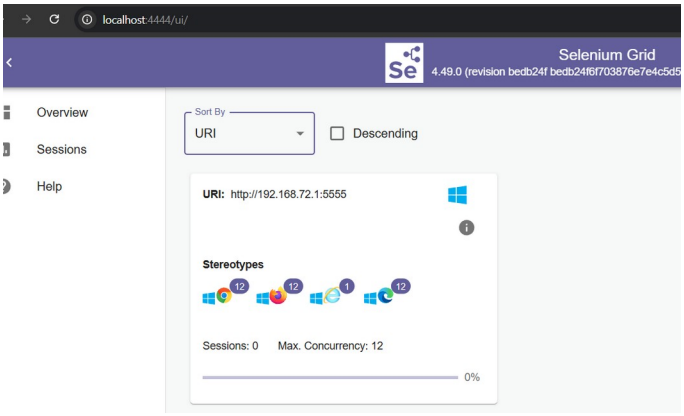
4. Wait for the Node to start. You will see:

Started Selenium node 4.49.0 (revision ...): http://192.168.72.1:5555

PART 7: Verify Node Registration

1. Go back to http://localhost:4444/ui/ in your browser.
2. Refresh (F5).
3. You should now see the Node listed with:

URI: http://192.168.72.1:5555
Chrome, Firefox, Edge capabilities shown as icons
Sessions: 0
Max Concurrency: 12
Take a screenshot of this page. This proves Hub-Node architecture is working.



PART 8: Create the Eclipse Maven Project
Group Id: com.stqa
Artifact Id: selenium-grid-practical4
Version: 1.0
Packaging: jar

1. Click **Finish**.

PART 9: Configure pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.stqa</groupId>
    <artifactId>selenium-grid-practical4</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.49.0</version>
        </dependency>
    </dependencies>
```

</project>

5. Press **Ctrl+S** to save.
6. .

PART 10: Fix the JRE to Use JDK 17
[jdk-17].

PART 11: Write the Java Test Class

1. In the Package Explorer, expand selenium-grid-practical4 → src/main/java.
2. Right-click on src/main/java → **New** → **Package**.
3. Name: com.stqa.grid. Click **Finish**.
4. Right-click on com.stqa.grid → **New** → **Class**.
5. Name: GridCrossBrowserTest.

```
package com.stqa.grid;

import java.net.URL;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.MutableCapabilities;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.remote.RemoteWebDriver;

public class GridCrossBrowserTest {

    private static final String GRID_URL = "http://localhost:4444";
    private static final String TEST_URL = "https://example.com/";

    public static void main(String[] args) {
        System.out.println("Starting Cross-Browser Test through Selenium Grid...\n");
        runTest("Chrome", new ChromeOptions());
        runTest("Firefox", new FirefoxOptions());
    }

    private static void runTest(String browserName, MutableCapabilities options) {
        WebDriver driver = null;
        try {
            options.setCapability("se:name", "STQA Practical 4 - " + browserName);
            driver = new RemoteWebDriver(new URL(GRID_URL), options);
            driver.get(TEST_URL);

            String title = driver.getTitle();
            String currentUrl = driver.getCurrentUrl();

            System.out.println("----------------------------------------");
            System.out.println("Browser : " + browserName);
            System.out.println("Title   : " + title);
            System.out.println("URL     : " + currentUrl);
```

```
            if (title.equals("Example Domain")) {
                System.out.println("Result  : PASS");
            } else {
                System.out.println("Result  : FAIL");
            }
        } catch (Exception e) {
            System.out.println("----------------------------------------");
            System.out.println("Browser : " + browserName);
            System.out.println("Result  : FAIL");
            System.out.println("Error   : " + e.getMessage());
        } finally {
            if (driver != null) {
                driver.quit();
            }
        }
    }
}
```

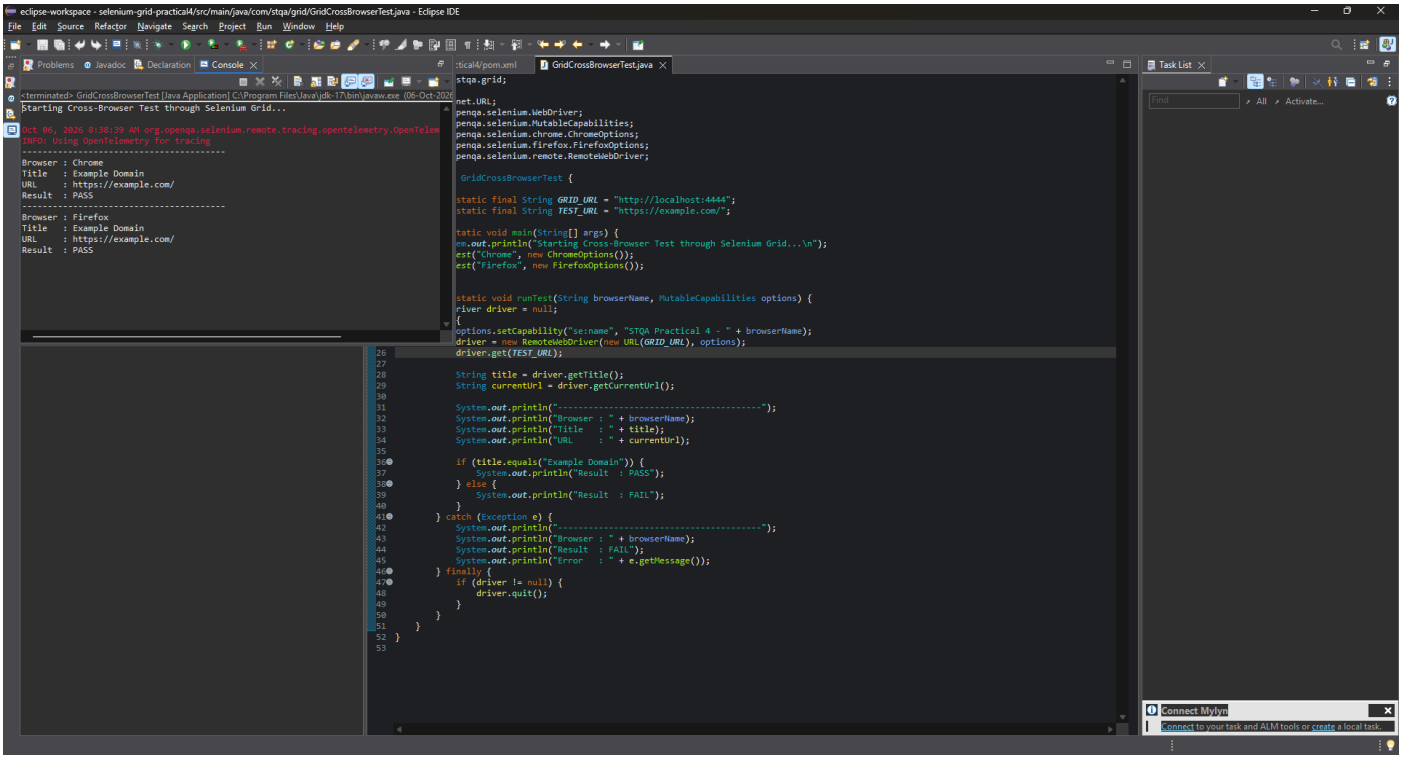
PART 12: Run the Test

1. In Eclipse, right-click on GridCrossBrowserTest.java → **Run As** → **Java Application**.

PART 14: Comparison Table

Prepare this in your journal based on what you observed:

Test Parameter	Chrome	Firefox
Browser session created	Pass	Pass
Page opened	Pass	Pass
Title verification	Example Domain	Example Domain
URL verification	https://example.com/	https://example.com/
Overall result	**Pass**	**Pass**


eclipse-workspace - selenium-grid-practical4/src/main/java/com/stqa/grid/GridCrossBrowserTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Problems Javadoc Declaration Console
<terminated> GridCrossBrowserTest [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (06-Oct-2026
Starting Cross-Browser Test through Selenium Grid...
Oct 06, 2026 8:38:39 AM org.openqa.selenium.remote.tracing.opentelemetry.OpenTelem
INFO: Using OpenTelemetry for tracing
Browser : Chrome
Title : Example Domain
URL : https://example.com/
Result : PASS
Browser : Firefox
Title : Example Domain
URL : https://example.com/
Result : PASS
:tical4/pom.xml
GridCrossBrowserTest.java
stqa.grid;
net.URL;
penqa.selenium.WebDriver;
penqa.selenium.MutableCapabilities;
penqa.selenium.chrome.ChromeOptions;
penqa.selenium.firefox.FirefoxOptions;
penqa.selenium.remote.RemoteWebDriver;
GridCrossBrowserTest {
static final String GRID_URL = "http://localhost:4444";
static final String TEST_URL = "https://example.com/";
tatic void main(String[] args) {
em.out.println("Starting Cross-Browser Test through Selenium Grid...\n");
est("Chrome", new ChromeOptions());
est("Firefox", new FirefoxOptions());
static void runTest(String browserName, MutableCapabilities options) {
river driver = null;
{
options.setCapability("se:name", "STQA Practical 4 - " + browserName);
driver = new RemoteWebDriver(new URL(GRID_URL), options);
driver.get(TEST_URL);
String title = driver.getTitle();
String currentUrl = driver.getCurrentUrl();
System.out.println("--");
System.out.println("Browser : " + browserName);
System.out.println("Title : " + title);
System.out.println("URL : " + currentUrl);
if (title.equals("Example Domain")) {
System.out.println("Result : PASS");
} else {
System.out.println("Result : FAIL");
}
} catch (Exception e) {
System.out.println("--");
System.out.println("Browser : " + browserName);
System.out.println("Result : FAIL");
System.out.println("Error : " + e.getMessage());
} finally {
if (driver != null) {
driver.quit();
}
}
}
}
Task List
Find
All
Activate...
Connect Mylyn
Connect to your task and ALM tools or create a local task.

PRACTICAL 5: Login Automation and Validation Using Selenium WebDriver

We'll use **SauceDemo** again because we already know it, and now we'll see how much easier it is to automate with WebDriver compared to Selenium IDE. The React login that broke Selenium IDE works perfectly with WebDriver because WebDriver uses native OS-level keyboard events, not the broken initKeyEvent method.

We already have JDK 17, Eclipse, Maven, and Selenium dependencies set up from Practical 4.

Requirements

JDK 17 (already installed)
Eclipse IDE (already installed)
Selenium Java 4.49.0 (already in Maven)
SauceDemo credentials: standard_user / secret_sauce

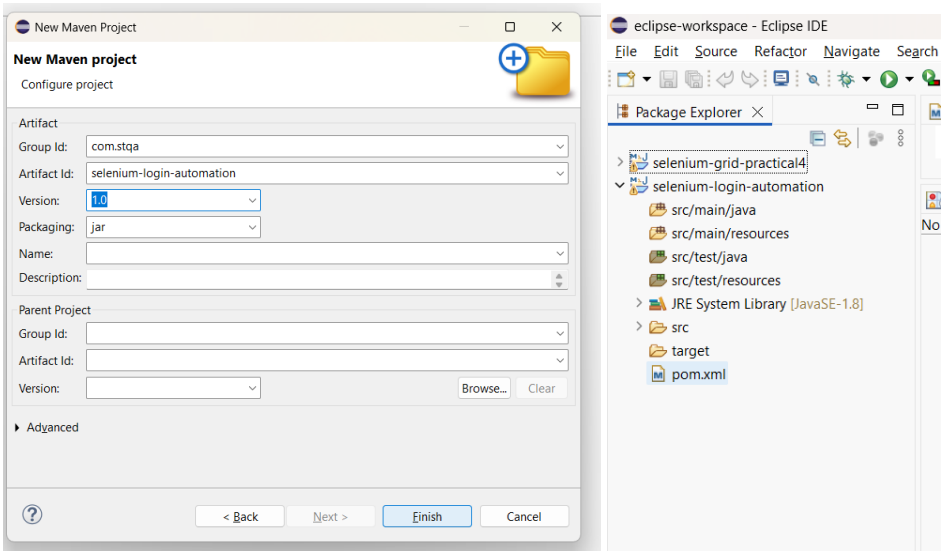
Theory:

Selenium WebDriver is a library that allows testers to control a browser through code. Unlike Selenium IDE which records and plays back actions, WebDriver lets you write reusable test scripts in languages like Java, Python, or C#. Assertions are used to verify that the actual result matches the expected result. Common assertions include checking the current URL, verifying that certain text is present, and checking if specific elements are visible on the page. If an assertion fails, the test fails and the failure is logged.

PART 1: Create a New Maven Project

Group Id: com.stqa
Artifact Id: selenium-login-automation
Version: 1.0
Packaging: jar

1. Click **Finish**.



PART 2: Configure the pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.stqa</groupId>
```

```
    <artifactId>selenium-login-automation</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.49.0</version>
        </dependency>
    </dependencies>
</project>
```
.

PART 3: Fix JRE to Use JDK 17

PART 4: Create the Test Class

1. Right-click src/main/java → **New** → **Package**.
2. Name: com.stqa.login. Click **Finish**.
3. Right-click on the new package → **New** → **Class**.
4. Name: SauceDemoLoginTest.
5. Check public static void main(String[] args). Click **Finish**.

```
package com.stqa.login;

import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class SauceDemoLoginTest {

    public static void main(String[] args) {



java.util.logging.Logger.getLogger("org.openqa.selenium.devtools.CdpVersionFinder").setLevel(ja
va.util.logging.Level.OFF);
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--remote-allow-origins=*");
```

```
WebDriver driver = new ChromeDriver(options);

try {
    driver.manage().window().maximize();
    driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

    // Step 1: Open SauceDemo
    driver.get("https://www.saucedemo.com/");
    System.out.println("Opened SauceDemo login page.");

    // Step 2: Enter username
    WebElement usernameField = driver.findElement(By.id("user-name"));
    usernameField.clear();
    usernameField.sendKeys("standard_user");

    // Step 3: Enter password
    WebElement passwordField = driver.findElement(By.id("password"));
    passwordField.clear();
    passwordField.sendKeys("secret_sauce");

    // Step 4: Click Login
    WebElement loginButton = driver.findElement(By.id("login-button"));
    loginButton.click();
    System.out.println("Clicked Login button.");

    Thread.sleep(2000);

    // ====================================================
    // ASSERTION 1: URL Validation
    // ====================================================
    String actualUrl = driver.getCurrentUrl();
    if (actualUrl.contains("inventory.html")) {
        System.out.println("ASSERTION 1 (URL)       : PASS  -> " + actualUrl);
    } else {
        System.out.println("ASSERTION 1 (URL)       : FAIL  -> " + actualUrl);
    }

    // ====================================================
    // ASSERTION 2: Title Validation
    // ====================================================
    WebElement productsTitle = driver.findElement(By.className("title"));
    String actualTitle = productsTitle.getText();
    if (actualTitle.equals("Products")) {
        System.out.println("ASSERTION 2 (Title)     : PASS  -> " + actualTitle);
    } else {
        System.out.println("ASSERTION 2 (Title)     : FAIL  -> " + actualTitle);
    }
```

```
        // ================================================
        // ASSERTION 3: Logout Link Visibility Check
        // ================================================
        // Click hamburger menu using JavaScript (bypasses all click interception)
        WebElement menuButton = driver.findElement(By.id("react-burger-menu-btn"));
        JavascriptExecutor js = (JavascriptExecutor) driver;
        js.executeScript("arguments[0].click();", menuButton);
        System.out.println("Clicked hamburger menu.");

        // Wait for the logout link to be present in the DOM (not necessarily "visible" in Selenium's
view)
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        WebElement logoutLink = wait.until(
                ExpectedConditions.presenceOfElementLocated(By.id("logout_sidebar_link")));

        if (logoutLink.isDisplayed()) {
            System.out.println("ASSERTION 3 (Logout)    : PASS  -> Logout link is visible");
        } else {
            System.out.println("ASSERTION 3 (Logout)    : FAIL  -> Logout link not visible");
        }

        // ================================================
        // BONUS: Perform actual logout via JavaScript click
        // ================================================
        js.executeScript("arguments[0].click();", logoutLink);
        Thread.sleep(2000);

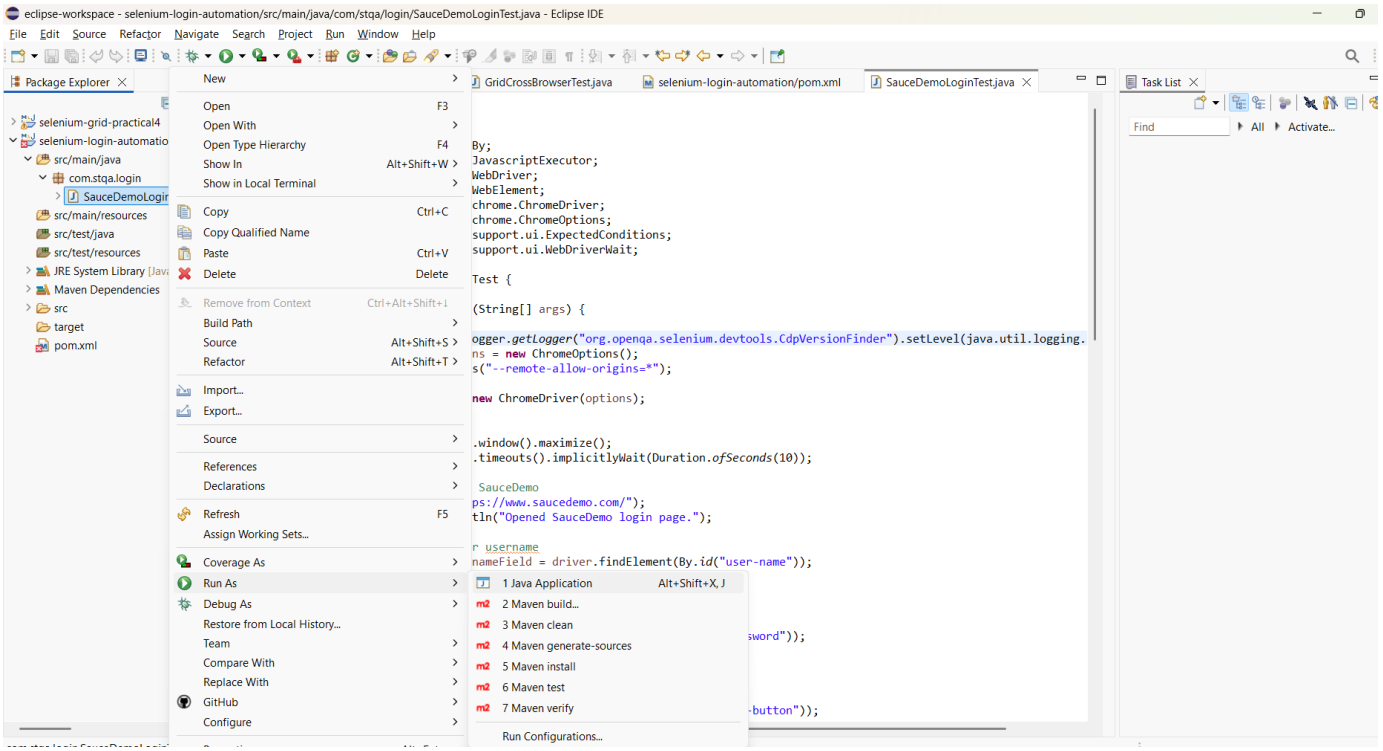
        String afterLogoutUrl = driver.getCurrentUrl();
        if (afterLogoutUrl.equals("https://www.saucedemo.com/")) {
            System.out.println("BONUS (After Logout)    : PASS  -> Back on login page");
        } else {
            System.out.println("BONUS (After Logout)    : FAIL  -> " + afterLogoutUrl);
        }

    } catch (Exception e) {
        System.out.println("Test encountered an error: " + e.getMessage());
    } finally {
        driver.quit();
        System.out.println("\nBrowser closed. Test complete.");
    }
  }
}
```

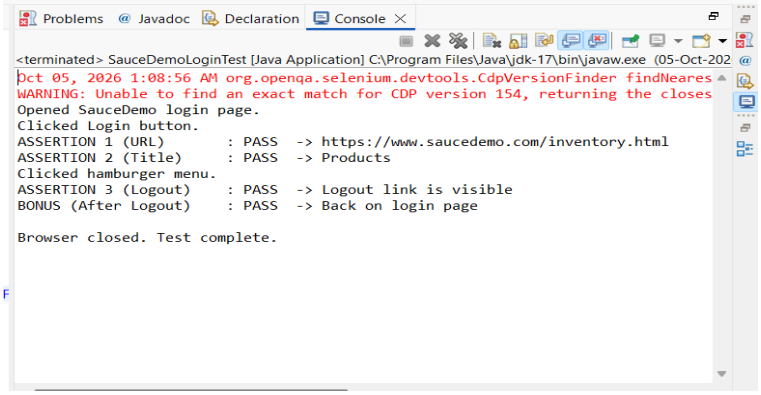
7. Press **Ctrl+S** to save. Check that there are no red underlines.

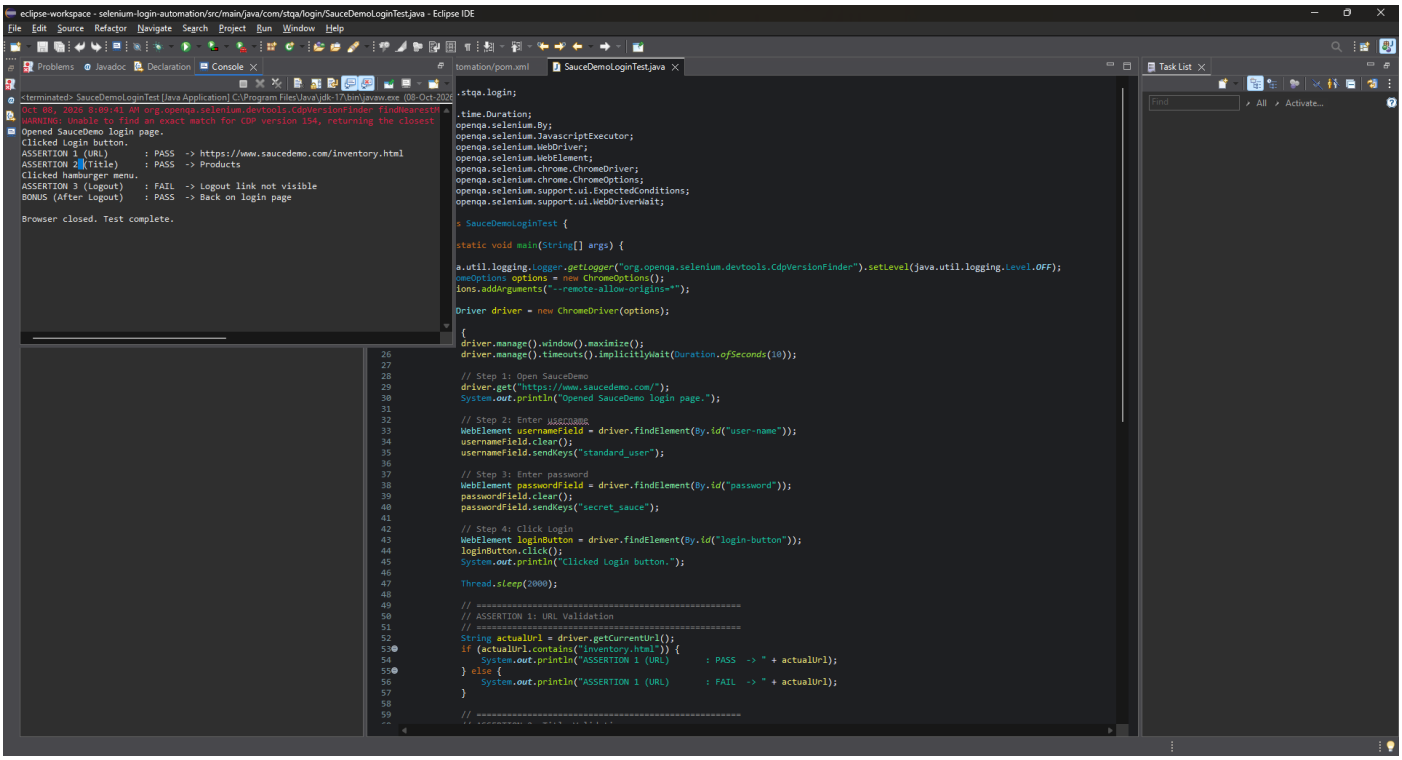
PART 5: Run the Test

1. Click **Run As → Java Application**.



2. Look at the **Console** tab in Eclipse. You should see:





PRACTICAL6: Web Element Interaction and Synchronization Handling

Aim: To develop a script to automate interaction with various web elements (textbox, radio button, checkbox, dropdown, alert, and frames) using Selenium WebDriver, and implement implicit and explicit waits for synchronization handling.

Requirements

- JDK 17 (already set up)
- Eclipse IDE (already set up)
- Selenium Java 4.49.0 (already in Maven)
- Google Chrome (already installed)
- Internet connection

No new installation needed. You'll reuse everything from Practical 5.

Theory:

Selenium WebDriver interacts with different HTML elements using different methods. Textboxes and buttons use sendKeys() and click(). Radio buttons and checkboxes also use click(), but their state is verified using isSelected().

JavaScript alerts, prompts, and confirms are not part of the page DOM — they are browser-level dialogs. To handle them, Selenium must switch context using driver.switchTo().alert().

Iframes are separate HTML documents embedded inside the parent page. Selenium cannot see the elements inside an iframe until it switches context using driver.switchTo().frame()..

Synchronization handles the fact that page elements do not appear instantly. An **implicit wait** is set globally with driver.manage().timeouts().implicitlyWait(). It applies to every findElement call and tells Selenium to poll the DOM for up to a specified duration when an element is not immediately available.

PART 1: Create a New Maven Project

Same as before, just a new project name.

Group Id: com.stqa
Artifact Id: selenium-webelement-practical6
Version: 1.0
Packaging: jar

PART 2: Configure pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.stqa</groupId>
    <artifactId>selenium-webelement-practical6</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
```

```
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.49.0</version>
        </dependency>
    </dependencies>
</project>
```

PART 3: Fix JRE to JDK 17

PART 4: Create the Test Class

1. Right-click src/main/java → **New** → **Package**.
2. Name: com.stqa.webelement. Click **Finish**.
3. Right-click on the package → **New** → **Class**.
4. Name: WebElementInteractionTest.
5. Check public static void main(String[] args). Click **Finish**.

```
package com.stqa.webelement;

import java.time.Duration;
import java.util.List;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.*;
import org.openqa.selenium.support.ui.*;

public class WebElementInteractionTest {

    public static void main(String[] args) throws InterruptedException {
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--remote-allow-origins=*");
        WebDriver driver = new ChromeDriver(options);
        driver.manage().window().maximize();

        // Implicit wait — applies globally to every findElement
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        // Explicit wait — used for a specific condition
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));

        try {
            // 1. TEXTBOX
            driver.get("https://the-internet.herokuapp.com/inputs");
            WebElement txt = driver.findElement(By.tagName("input"));
            txt.sendKeys("12345");
            System.out.println("1. Textbox  : " + (txt.getAttribute("value").equals("12345") ? "PASS" :
"FAIL"));

            // 2. RADIO BUTTON
            driver.get("https://demoqa.com/radio-button");
            Thread.sleep(1500);
```

```
        driver.findElement(By.xpath("//label[text()='Yes']")).click();
        System.out.println("2. Radio    : " + (driver.findElement(By.id("yesRadio")).isSelected() ?
"PASS" : "FAIL"));

        // 3. CHECKBOX (toggle two boxes)
        driver.get("https://the-internet.herokuapp.com/checkboxes");
        List<WebElement> cbs = driver.findElements(By.cssSelector("input[type='checkbox']"));
        if (!cbs.get(0).isSelected()) cbs.get(0).click();
        if (cbs.get(1).isSelected())  cbs.get(1).click();
        System.out.println("3. Checkbox : " + (cbs.get(0).isSelected() && !cbs.get(1).isSelected() ?
"PASS" : "FAIL"));

        // 4. DROPDOWN (Select class)
        driver.get("https://the-internet.herokuapp.com/dropdown");
        Select select = new Select(driver.findElement(By.id("dropdown")));
        select.selectByVisibleText("Option 1");
        select.selectByIndex(2);
        System.out.println("4. Dropdown : " +
(select.getFirstSelectedOption().getText().equals("Option 2") ? "PASS" : "FAIL"));

        // 5. ALERT (accept OK, dismiss Cancel)
        driver.get("https://the-internet.herokuapp.com/javascript_alerts");
        driver.findElement(By.xpath("//button[text()='Click for JS Alert']")).click();
        Thread.sleep(500);
        driver.switchTo().alert().accept();
        Thread.sleep(500);
        String r1 = driver.findElement(By.id("result")).getText();
        driver.findElement(By.xpath("//button[text()='Click for JS Confirm']")).click();
        Thread.sleep(500);
        driver.switchTo().alert().dismiss();
        Thread.sleep(500);
        String r2 = driver.findElement(By.id("result")).getText();
        System.out.println("5. Alert    : " + (r1.contains("successfully clicked") &&
r2.contains("Cancel") ? "PASS" : "FAIL"));

        // 6. IFRAME (switch in and back out)
        driver.get("https://the-internet.herokuapp.com/iframe");
        driver.switchTo().frame("mce_0_ifr");
        String inner = driver.findElement(By.id("tinymce")).getText();
        driver.switchTo().defaultContent();
        String outer = driver.findElement(By.tagName("h3")).getText();
        System.out.println("6. Iframe   : " + (inner.contains("Your content") && outer.contains("An
iFrame") ? "PASS" : "FAIL"));

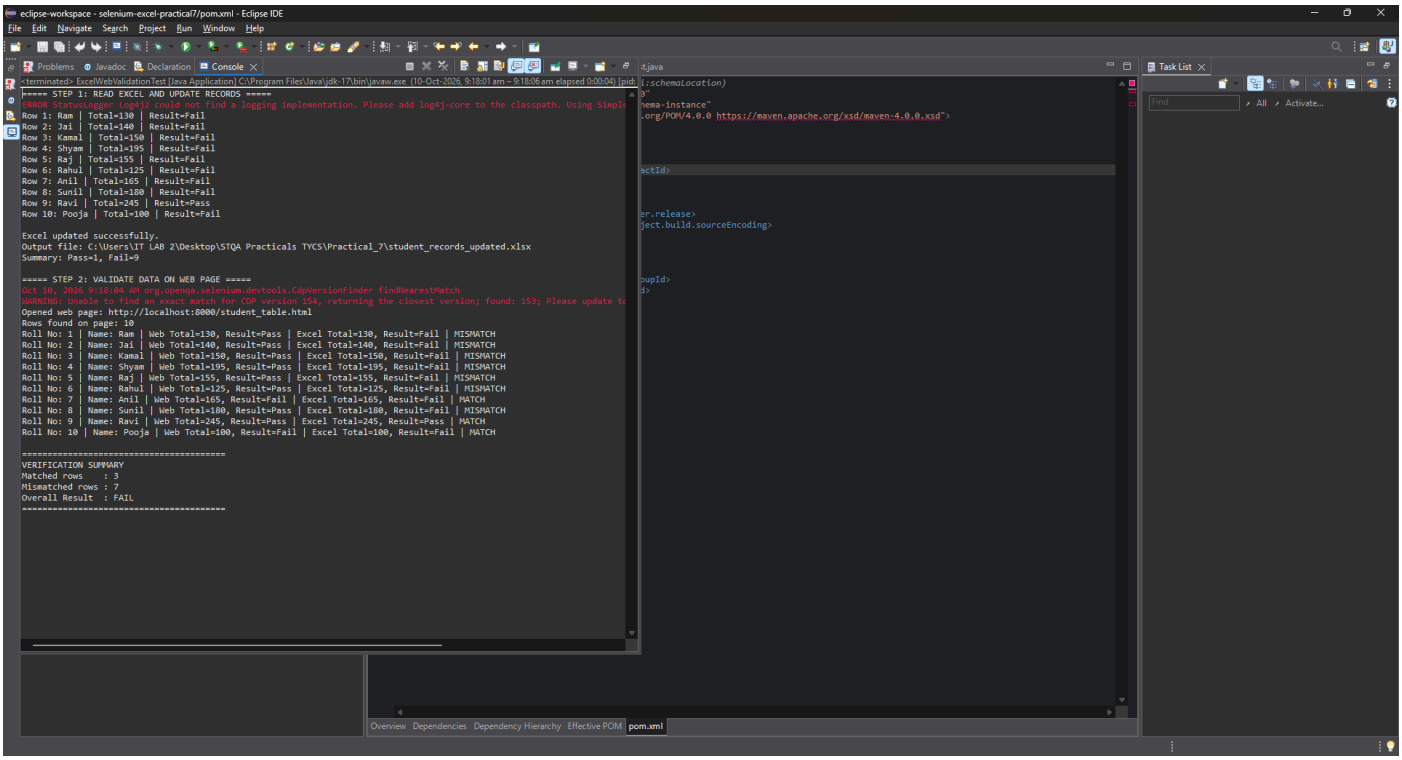
        // 7. EXPLICIT WAIT (element appears after ~5 sec)
        driver.get("https://the-internet.herokuapp.com/dynamic_loading/2");
        driver.findElement(By.xpath("//button[text()='Start']")).click();
        WebElement hello = wait.until(ExpectedConditions.visibilityOfElementLocated(
                By.xpath("//h4[text()='Hello World!']")));
```

```
            System.out.println("7. ExpWait  : " + (hello.getText().equals("Hello World!") ? "PASS" : "FAIL"));

            System.out.println("\nAll element interactions completed.");

        } catch (Exception e) {
            System.out.println("Test error: " + e.getMessage());
        } finally {
            driver.quit();
        }
    }
}
```

PART 5: Run the Test



.

Practical 7:

```
<terminated> ExcelWebValidationTest [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe  (05-Oct-2026, 8:24:49 pm – 8:25:03 pm elapsed: 0:00:13.887) [pid: 16
===== STEP 1: READ EXCEL AND UPDATE RECORDS =====

ERROR StatusLogger Log4j2 could not find a logging implementation. Please add log4j-core to the classpath. Using SimpleLogger
Row 1: Ram | Total=130 | Result=Pass
Row 2: Jai | Total=140 | Result=Pass
Row 3: Kamal | Total=150 | Result=Pass
Row 4: Shyam | Total=195 | Result=Pass
Row 5: Raj | Total=155 | Result=Pass
Row 6: Rahul | Total=125 | Result=Pass
Row 7: Anil | Total=165 | Result=Fail
Row 8: Sunil | Total=180 | Result=Pass
Row 9: Ravi | Total=245 | Result=Pass
Row 10: Pooja | Total=100 | Result=Fail

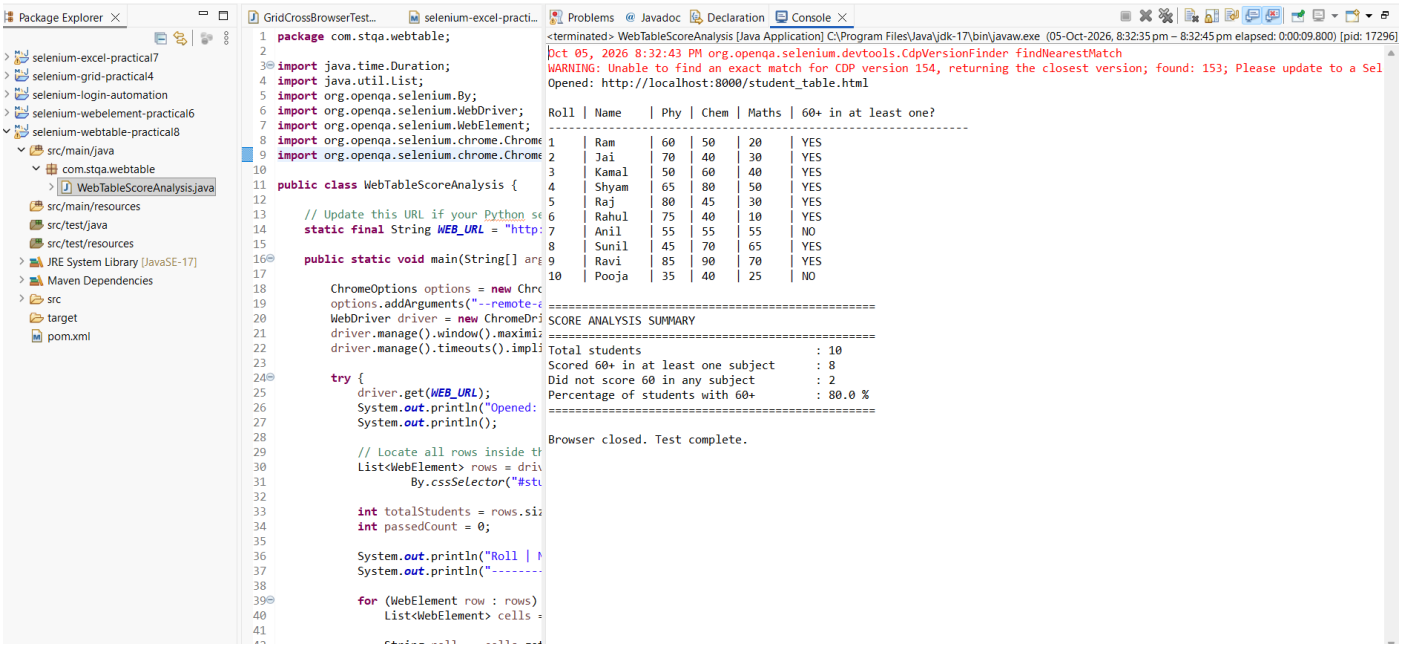
Excel updated. Output: C:\Users\Vicky Kumar\OneDrive\Desktop\STQA_Practicals TYCS\Practical_7\student_records_updated.xlsx
Summary: Pass=8 Fail=2

===== STEP 2: VALIDATE DATA ON WEB PAGE =====

Oct 05, 2026 8:25:00 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 154, returning the closest version; found: 153; Please update to a Sel
Opened web page: http://localhost:8000/student_table.html
Rows found on page: 10
Row 1 | Ram | Web[Total=130, Result=Pass] | Excel[Total=130, Result=Pass] | MATCH
Row 2 | Jai | Web[Total=140, Result=Pass] | Excel[Total=140, Result=Pass] | MATCH
Row 3 | Kamal | Web[Total=150, Result=Pass] | Excel[Total=150, Result=Pass] | MATCH
Row 4 | Shyam | Web[Total=195, Result=Pass] | Excel[Total=195, Result=Pass] | MATCH
Row 5 | Raj | Web[Total=155, Result=Pass] | Excel[Total=155, Result=Pass] | MATCH
Row 6 | Rahul | Web[Total=125, Result=Pass] | Excel[Total=125, Result=Pass] | MATCH
Row 7 | Anil | Web[Total=165, Result=Fail] | Excel[Total=165, Result=Fail] | MATCH
Row 8 | Sunil | Web[Total=180, Result=Pass] | Excel[Total=180, Result=Pass] | MATCH
Row 9 | Ravi | Web[Total=245, Result=Pass] | Excel[Total=245, Result=Pass] | MATCH
Row 10 | Pooja | Web[Total=100, Result=Fail] | Excel[Total=100, Result=Fail] | MATCH

========================================
Verification Summary:
  Matched rows    : 10
  Mismatched rows : 0
  Overall Result  : PASS
========================================
```

Practical no. 8:



PRACTICAL 7: Data-Driven Testing Using Excel Integration

Aim: To write a program using Selenium WebDriver and Apache POI to update 10 student records in an Excel file and validate the changes on a web application.

Requirements:

JDK 17 installed
Eclipse IDE
Selenium Java 4.49.0
Apache POI 5.2.5
Python (for serving the HTML locally)
Google Chrome
Microsoft Excel
Internet connection

Theory:

Data-driven testing is a testing approach where the test data is stored externally — in files like Excel, CSV, or a database — instead of being hardcoded inside the Java code. The same test logic runs multiple times, once per data row. This makes tests easier to maintain: if you need to change the test data, you edit the Excel file, not the source code.

Apache POI is a Java library for reading and writing Microsoft Office files. For Excel workbooks, the library provides the classes Workbook, Sheet, Row, and Cell. The implementation XSSFWorkbook handles .xlsx files, while HSSFWorkbook handles the older .xls format. Numeric cells are read with getNumericCellValue() and text cells with getStringCellValue(). New values are written using setCellValue(). File reading and writing are handled by FileInputStream and FileOutputStream.

The **web validation step** uses Selenium WebDriver to open an HTML page containing a table, extract each row using CSS selectors, and compare the values against the Excel data programmatically.

Procedure

Step 1: Prepare the Input Excel File

Roll No	Name	Physics	Chemistry	Maths
1	Ram	60	50	20
2	Jai	70	40	30
3	Kamal	50	60	40
4	Shyam	65	80	50
5	Raj	80	45	30
6	Rahul	75	40	10
7	Anil	55	55	55
8	Sunil	45	70	65
9	Ravi	85	90	70

Roll No	Name	Physics	Chemistry	Maths
10	Pooja	35	40	25

Step 2: Create the HTML File for Web Validation

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Records</title>
    <style>
        body { font-family: Arial; padding: 20px; }
        table { border-collapse: collapse; width: 80%; }
        th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
        th { background-color: #f2f2f2; }
    </style>
</head>
<body>
    <h2>Student Records - Admin Panel</h2>
    <table id="studentTable">
        <thead>
            <tr>
                <th>Roll No</th>
                <th>Name</th>
                <th>Physics</th>
                <th>Chemistry</th>
                <th>Maths</th>
                <th>Total</th>
                <th>Result</th>
            </tr>
        </thead>
        <tbody>
            <tr><td>1</td><td>Ram</td><td>60</td><td>50</td><td>20</td><td>130</
td><td>Pass</td></tr>
            <tr><td>2</td><td>Jai</td><td>70</td><td>40</td><td>30</td><td>140</td><td>Pass</
td></tr>
            <tr><td>3</td><td>Kamal</td><td>50</td><td>60</td><td>40</td><td>150</
td><td>Pass</td></tr>
            <tr><td>4</td><td>Shyam</td><td>65</td><td>80</td><td>50</td><td>195</
td><td>Pass</td></tr>
            <tr><td>5</td><td>Raj</td><td>80</td><td>45</td><td>30</td><td>155</td><td>Pass</
td></tr>
            <tr><td>6</td><td>Rahul</td><td>75</td><td>40</td><td>10</td><td>125</
td><td>Pass</td></tr>
            <tr><td>7</td><td>Anil</td><td>55</td><td>55</td><td>55</td><td>165</td><td>Fail</
td></tr>
            <tr><td>8</td><td>Sunil</td><td>45</td><td>70</td><td>65</td><td>180</
td><td>Pass</td></tr>
            <tr><td>9</td><td>Ravi</td><td>85</td><td>90</td><td>70</td><td>245</
td><td>Pass</td></tr>
```

```
        <tr><td>10</td><td>Pooja</td><td>35</td><td>40</td><td>25</td><td>100</td><td>Fail</td></tr>
      </tbody>
   </table>
</body>
</html>
```

Step 3: Start the Local Web Server

python -m http.server 8000

Step 4: Create the Maven Project in Eclipse

Group Id: com.stqa
Artifact Id: selenium-excel-practical7
Version: 1.0
Packaging: jar

Step 5: Configure pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>

   <groupId>com.stqa</groupId>
   <artifactId>selenium-excel-practical7</artifactId>
   <version>1.0</version>

   <properties>
      <maven.compiler.release>17</maven.compiler.release>
      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
   </properties>

   <dependencies>
      <dependency>
         <groupId>org.seleniumhq.selenium</groupId>
         <artifactId>selenium-java</artifactId>
         <version>4.49.0</version>
      </dependency>

      <dependency>
         <groupId>org.apache.poi</groupId>
         <artifactId>poi</artifactId>
         <version>5.2.5</version>
      </dependency>
      <dependency>
         <groupId>org.apache.poi</groupId>
         <artifactId>poi-ooxml</artifactId>
         <version>5.2.5</version>
      </dependency>
```

```
    </dependencies>
</project>
```

Step 7: Create the Package and Class

1. Expand src/main/java → right-click → **New** → **Package**.
2. Name: com.stqa.excel. Click **Finish**.
3. Right-click the package → **New** → **Class**.
4. Name: ExcelWebValidationTest.

Step 8: Write the Java Code

```
package com.stqa.excel;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.time.Duration;
import java.util.List;
import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class ExcelWebValidationTest {

    // Update these three paths to match your folder
    static final String INPUT_EXCEL  = "C:\\Users\\VAbr\\OneDrive\\Desktop\\STQA_Practicals
TYCS\\Practical_7\\student_records.xlsx";
    static final String OUTPUT_EXCEL = "C:\\Users\\VAbr\\OneDrive\\Desktop\\STQA_Practicals
TYCS\\Practical_7\\student_records_updated.xlsx";
    static final String WEB_URL      = "http://localhost:8000/student_table.html";

    public static void main(String[] args) throws Exception {

        System.out.println("===== STEP 1: READ EXCEL AND UPDATE RECORDS =====\n");

        FileInputStream fis = new FileInputStream(INPUT_EXCEL);
        Workbook workbook = new XSSFWorkbook(fis);
        Sheet sheet = workbook.getSheetAt(0);

        int lastRow = sheet.getLastRowNum();
        Row header = sheet.getRow(0);
        int lastCol = header.getLastCellNum();

        header.createCell(lastCol).setCellValue("Total");
        header.createCell(lastCol + 1).setCellValue("Result");

        int passCount = 0, failCount = 0;
```

```
for (int i = 1; i <= lastRow; i++) {
    Row row = sheet.getRow(i);
    String name = row.getCell(1).getStringCellValue();
    int phy  = (int) row.getCell(2).getNumericCellValue();
    int chem = (int) row.getCell(3).getNumericCellValue();
    int math = (int) row.getCell(4).getNumericCellValue();

    int total = phy + chem + math;
    String result = (phy >= 60 || chem >= 60 || math >= 60) ? "Pass" : "Fail";

    if (result.equals("Pass")) passCount++; else failCount++;

    row.createCell(lastCol).setCellValue(total);
    row.createCell(lastCol + 1).setCellValue(result);

    System.out.println("Row " + i + ": " + name
            + " | Total=" + total + " | Result=" + result);
}

FileOutputStream fos = new FileOutputStream(OUTPUT_EXCEL);
workbook.write(fos);
fos.close();
workbook.close();
fis.close();

System.out.println("\nExcel updated. Output: " + OUTPUT_EXCEL);
System.out.println("Summary: Pass=" + passCount + " Fail=" + failCount);

System.out.println("\n===== STEP 2: VALIDATE DATA ON WEB PAGE =====\n");

ChromeOptions options = new ChromeOptions();
options.addArguments("--remote-allow-origins=*");
WebDriver driver = new ChromeDriver(options);
driver.manage().window().maximize();
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

try {
    driver.get(WEB_URL);
    System.out.println("Opened web page: " + WEB_URL);

    List<WebElement> rows = driver.findElements(
            By.cssSelector("#studentTable tbody tr"));
    System.out.println("Rows found on page: " + rows.size());

    FileInputStream fis2 = new FileInputStream(OUTPUT_EXCEL);
    Workbook wb2 = new XSSFWorkbook(fis2);
    Sheet sheet2 = wb2.getSheetAt(0);
```

```
            int matches = 0;
            int mismatches = 0;

            for (int i = 0; i < rows.size(); i++) {
                List<WebElement> cells = rows.get(i).findElements(By.tagName("td"));

                String webRoll   = cells.get(0).getText();
                String webName   = cells.get(1).getText();
                String webTotal  = cells.get(5).getText();
                String webResult = cells.get(6).getText();

                Row excelRow = sheet2.getRow(i + 1);
                String excelTotal  = String.valueOf((int) excelRow.getCell(5).getNumericCellValue());
                String excelResult = excelRow.getCell(6).getStringCellValue();

                boolean match = excelTotal.equals(webTotal) && excelResult.equals(webResult);
                if (match) matches++; else mismatches++;

                System.out.println("Row " + webRoll + " | " + webName
                        + " | Web[Total=" + webTotal + ", Result=" + webResult + "]"
                        + " | Excel[Total=" + excelTotal + ", Result=" + excelResult + "]"
                        + " | " + (match ? "MATCH" : "MISMATCH"));
            }

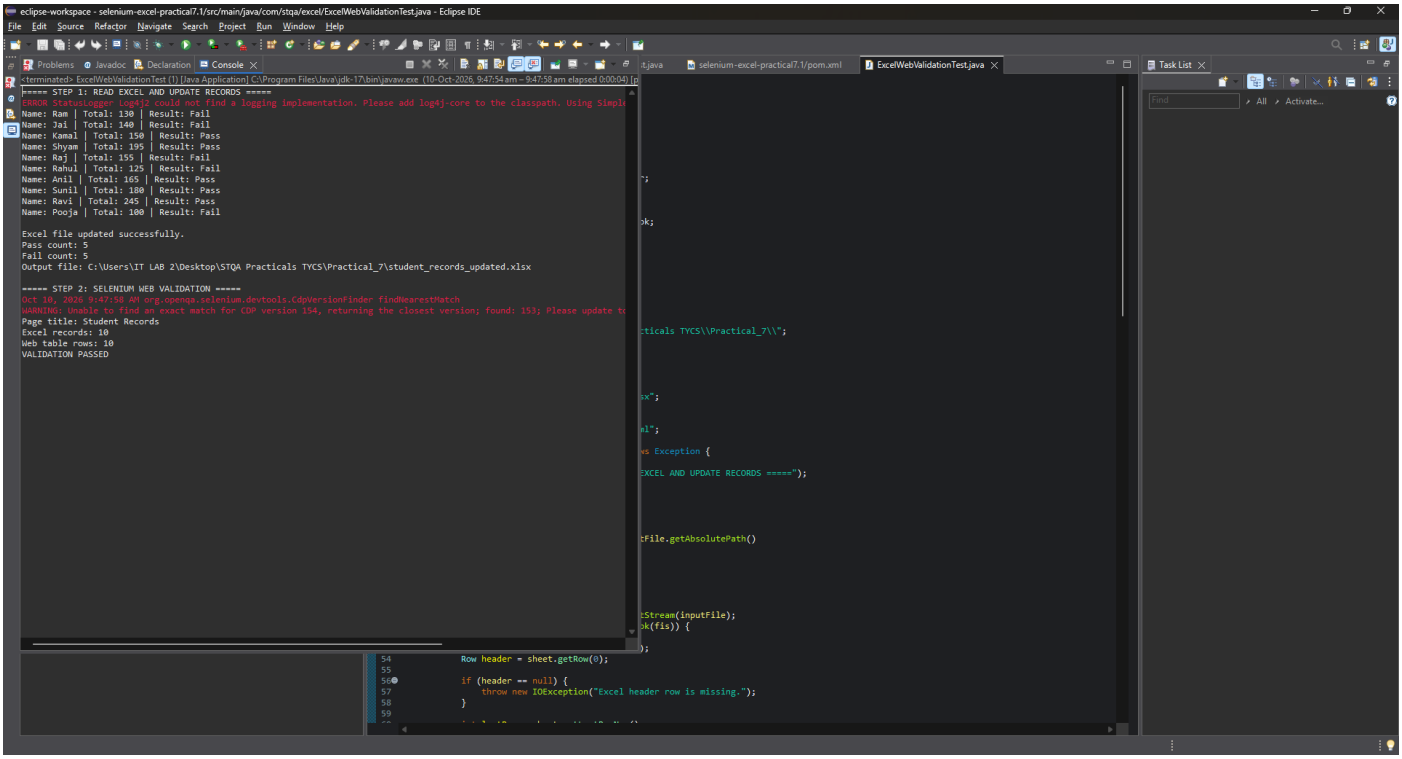
            wb2.close();
            fis2.close();

            System.out.println("\n========================================");
            System.out.println("Verification Summary:");
            System.out.println("  Matched rows    : " + matches);
            System.out.println("  Mismatched rows : " + mismatches);
            System.out.println("  Overall Result  : " + (mismatches == 0 ? "PASS" : "FAIL"));
            System.out.println("========================================");

        } finally {
            driver.quit();
        }
    }
}
```

Save with **Ctrl+S**.

Step 9: Run the Program


eclipse-workspace - selenium-excel-practical7.1/src/main/java/com/stqa/excel/ExcelWebValidationTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Problems Javadoc Declaration Console
<terminated> ExcelWebValidationTest (1) [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (10-Oct-2026, 9:47:54 am – 9:47:58 am elapsed 0:00:04)
===== STEP 1: READ EXCEL AND UPDATE RECORDS =====
ERROR StatusLogger Log4j2 could not find a logging implementation. Please add log4j-core to the classpath. Using Simple
Name: Ram | Total: 130 | Result: Fail
Name: Jai | Total: 140 | Result: Fail
Name: Kamal | Total: 150 | Result: Pass
Name: Shyam | Total: 195 | Result: Pass
Name: Raj | Total: 155 | Result: Fail
Name: Rahul | Total: 125 | Result: Fail
Name: Anil | Total: 165 | Result: Pass
Name: Sunil | Total: 180 | Result: Pass
Name: Ravi | Total: 245 | Result: Pass
Name: Pooja | Total: 100 | Result: Fail
Excel file updated successfully.
Pass count: 5
Fail count: 5
Output file: C:\Users\IT LAB 2\Desktop\STQA Practicals TYCS\Practical_7\student_records_updated.xlsx
===== STEP 2: SELENIUM WEB VALIDATION =====
Oct 10, 2026 9:47:58 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 154, returning the closest version; found: 153; Please update to
Page title: Student Records
Excel records: 10
Web table rows: 10
VALIDATION PASSED
selenium-excel-practical7.1/pom.xml
ExcelWebValidationTest.java
Task List
Row header = sheet.getRow(0);
if (header == null) {
throw new IOException("Excel header row is missing.");
}

PRACTICAL 8: Data Extraction and Score Analysis from Web Table

Aim: To automate the extraction of student marks from a web table using Selenium WebDriver and determine the number of students who scored 60 or above in at least one subject. Compute the total count and percentage.

Requirements

JDK 17 installed
Eclipse IDE
Selenium Java 4.49.0
Python (for serving the HTML locally)
Google Chrome

Theory:

A **web table** in HTML is built using the <table>, <thead>, <tbody>, <tr>, <th>, and <td> tags. Each <tr> element represents a row, and each <td> inside it is a cell. Selenium WebDriver can extract the contents of a table by locating all <tr> elements inside the <tbody> and then iterating through the <td> cells of each row.

Procedure

Step 1: Ensure the Local Web Server Is Running

http://localhost:8000/student_table.html

.

Step 2: Create the Maven Project in Eclipse

Group Id: com.stqa
Artifact Id: selenium-webtable-practical8
Version: 1.0
Packaging: jar

Step 3: Configure pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.stqa</groupId>
    <artifactId>selenium-webtable-practical8</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.49.0</version>
        </dependency>
```

```
    </dependencies>
</project>
```

Step 4: Configure the JRE to JDK 17

Step 5: Create the Package and Class

1. Expand selenium-webtable-practical8 → expand src/main/java.
2. Right-click on src/main/java → **New** → **Package**.
3. Name: com.stqa.webtable. Click **Finish**.
4. Right-click the package → **New** → **Class**.
5. Name: WebTableScoreAnalysis.

Step 6: Write the Java Code

```
package com.stqa.webtable;

import java.time.Duration;
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class WebTableScoreAnalysis {

    // Update this URL if your Python server is on a different port
    static final String WEB_URL = "http://localhost:8000/student_table.html";

    public static void main(String[] args) {

        ChromeOptions options = new ChromeOptions();
        options.addArguments("--remote-allow-origins=*");
        WebDriver driver = new ChromeDriver(options);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        try {
            driver.get(WEB_URL);
            System.out.println("Opened: " + WEB_URL);
            System.out.println();

            // Locate all rows inside the tbody (skips the header row)
            List<WebElement> rows = driver.findElements(
                    By.cssSelector("#studentTable tbody tr"));

            int totalStudents = rows.size();
            int passedCount = 0;
```

```
            System.out.println("Roll | Name    | Phy | Chem | Maths | 60+ in at least one?");
            System.out.println("-----------------------------------------------------------");

            for (WebElement row : rows) {
                List<WebElement> cells = row.findElements(By.tagName("td"));

                String roll  = cells.get(0).getText();
                String name  = cells.get(1).getText();
                int physics  = Integer.parseInt(cells.get(2).getText());
                int chem     = Integer.parseInt(cells.get(3).getText());
                int maths    = Integer.parseInt(cells.get(4).getText());

                boolean scored60 = (physics >= 60 || chem >= 60 || maths >= 60);
                if (scored60) passedCount++;

                System.out.printf("%-4s | %-7s | %-3d | %-4d | %-5d | %s%n",
                        roll, name, physics, chem, maths, scored60 ? "YES" : "NO");
            }

            double percentage = (passedCount * 100.0) / totalStudents;

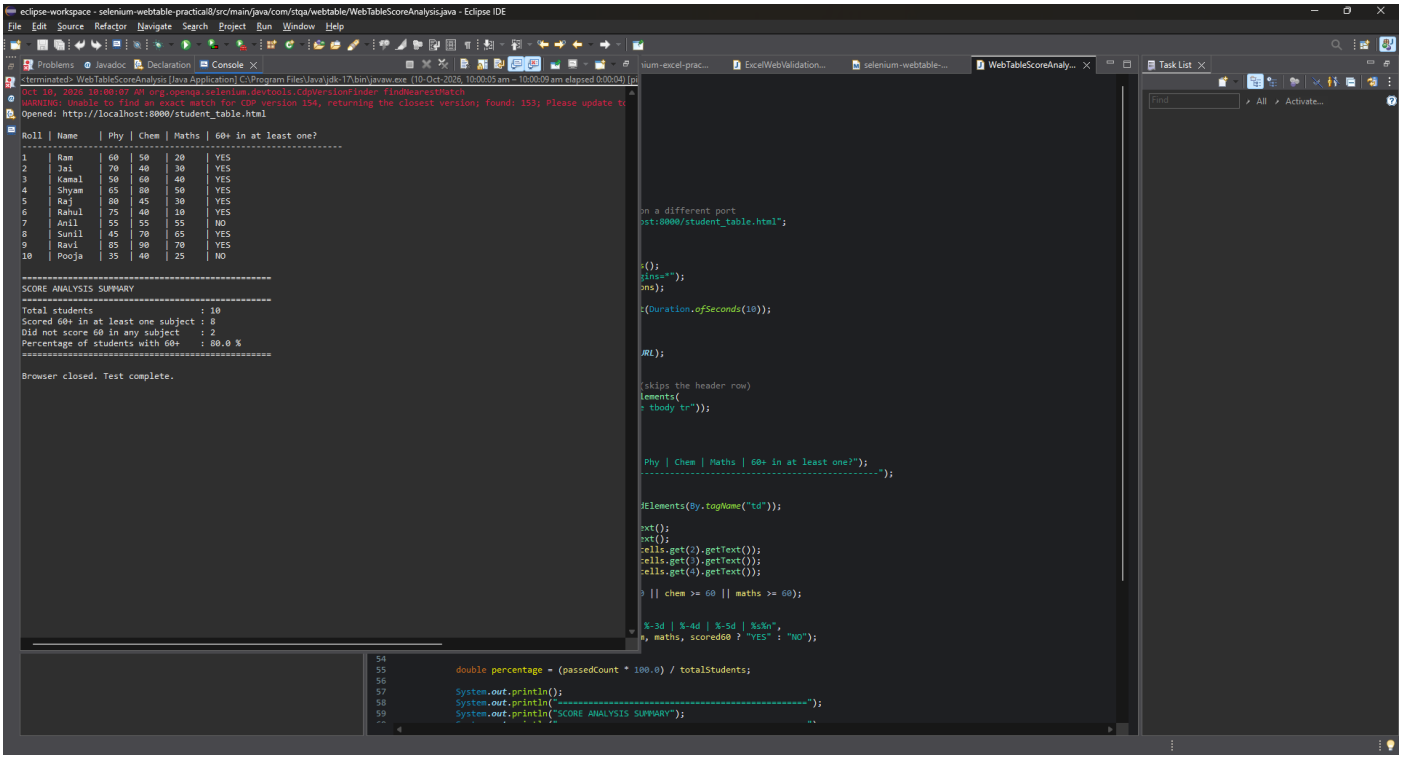
            System.out.println();
            System.out.println("================================================");
            System.out.println("SCORE ANALYSIS SUMMARY");
            System.out.println("================================================");
            System.out.println("Total students                    : " + totalStudents);
            System.out.println("Scored 60+ in at least one subject : " + passedCount);
            System.out.println("Did not score 60 in any subject    : " + (totalStudents - passedCount));
            System.out.println("Percentage of students with 60+    : " + percentage + " %");
            System.out.println("================================================");

        } catch (Exception e) {
            System.out.println("Test error: " + e.getMessage());
        } finally {
            driver.quit();
            System.out.println("\nBrowser closed. Test complete.");
        }
    }
}
```

Step 7: Run the Program

Result

The Selenium WebDriver program successfully extracted student marks from the HTML table, applied the score criterion, and computed the total count and percentage. Out of 10 students, 8 (80.0%) scored 60 or above in at least one subject.


eclipse-workspace - selenium-webtable-practical8/src/main/java/com/stqa/webtable/WebTableScoreAnalysis.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Problems Javadoc Declaration Console
<terminated> WebTableScoreAnalysis [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (10-Oct-2026, 10:00:05 am – 10:00:09 am elapsed 0:00:04)
Oct 10, 2026 10:00:07 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 154, returning the closest version; found: 153; Please update to
Opened: http://localhost:8000/student_table.html
Roll | Name | Phy | Chem | Maths | 60+ in at least one?
1 | Ram | 60 | 50 | 20 | YES
2 | Jai | 70 | 40 | 30 | YES
3 | Kamal | 50 | 60 | 40 | YES
4 | Shyam | 65 | 80 | 50 | YES
5 | Raj | 80 | 45 | 30 | YES
6 | Rahul | 75 | 40 | 10 | YES
7 | Anil | 55 | 55 | 55 | NO
8 | Sunil | 45 | 70 | 65 | YES
9 | Ravi | 85 | 90 | 70 | YES
10 | Pooja | 35 | 40 | 25 | NO
SCORE ANALYSIS SUMMARY
Total students : 10
Scored 60+ in at least one subject : 8
Did not score 60 in any subject : 2
Percentage of students with 60+ : 80.0 %
Browser closed. Test complete.
WebTableScoreAnaly...
Task List
double percentage = (passedCount * 100.0) / totalStudents;
System.out.println();
System.out.println("SCORE ANALYSIS SUMMARY");

PRACTICAL 9: Object Identification and Counting on Web Page

Aim: To write a program using Selenium WebDriver to identify and count the total number of objects present on a web page — including links, images, buttons, and input elements — and display categorized counts.

Requirements

1. JDK 17 installed
2. Eclipse IDE
3. Selenium Java 4.49.0

Theory:

Every HTML element on a web page has a tag name — <a> for links, <img> for images, <button> for buttons, <input> for input fields. Selenium WebDriver locates elements by tag name using By.tagName(). The findElements() method returns a list of all matching elements on the page. The size() method of the returned list gives the count.

.

Procedure

Step 1: Create the Maven Project in Eclipse

1. **Group Id:** com.stqa
2. **Artifact Id:** selenium-objectcount-practical9
3. **Version:** 1.0
4. **Packaging:** jar

Step 2: Configure pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.stqa</groupId>
    <artifactId>selenium-objectcount-practical9</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.49.0</version>
        </dependency>
    </dependencies>
</project>
```

Step 3: Configure the JRE to JDK 17

Step 4: Create the Package and Class

1. Expand selenium-objectcount-practical9 → expand src/main/java.

2. Right-click on src/main/java → **New** → **Package**.
3. Name: com.stqa.objectcount. Click **Finish**.
4. Right-click the package → **New** → **Class**.
5. Name: ObjectCountTest.

Step 5: Write the Java Code

```
package com.stqa.objectcount;

import java.time.Duration;
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class ObjectCountTest {

    // The website under test — The Internet (a stable automation demo site)
    static final String WEB_URL = "https://the-internet.herokuapp.com/";

    public static void main(String[] args) {

        ChromeOptions options = new ChromeOptions();
        options.addArguments("--remote-allow-origins=*");
        WebDriver driver = new ChromeDriver(options);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        try {
            driver.get(WEB_URL);
            System.out.println("Opened: " + WEB_URL);
            System.out.println();

            // Count different types of objects using findElements + size()
            List<WebElement> links   = driver.findElements(By.tagName("a"));
            List<WebElement> images  = driver.findElements(By.tagName("img"));
            List<WebElement> buttons = driver.findElements(By.tagName("button"));
            List<WebElement> inputs  = driver.findElements(By.tagName("input"));
            List<WebElement> headings = driver.findElements(By.tagName("h2"));

            int totalObjects = links.size() + images.size()
                    + buttons.size() + inputs.size() + headings.size();

            System.out.println("================== OBJECT COUNT SUMMARY
==================");
            System.out.println("Total objects found on the page : " + totalObjects);
            System.out.println("-----------------------------------------------------------");
            System.out.println("Links (<a>)                    : " + links.size());
```

```
        System.out.println("Images (<img>)                : " + images.size());
        System.out.println("Buttons (<button>)            : " + buttons.size());
        System.out.println("Input fields (<input>)        : " + inputs.size());
        System.out.println("Headings (<h2>)               : " + headings.size());
        System.out.println("-------------------------------------------------------------");

        // Show first few link texts as a sample
        System.out.println("\nFirst 5 link texts on the page:");
        int limit = Math.min(5, links.size());
        for (int i = 0; i < limit; i++) {
            String text = links.get(i).getText().trim();
            if (text.isEmpty()) text = "(no visible text)";
            System.out.println("  " + (i + 1) + ". " + text);
        }

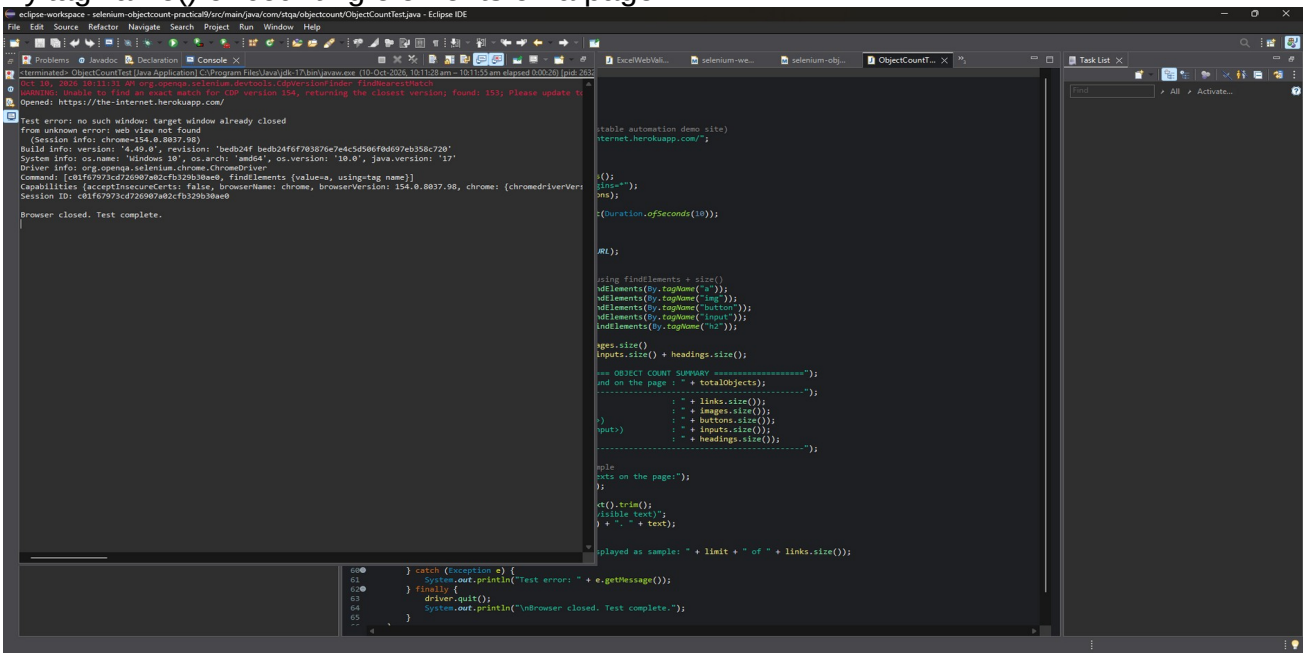
        System.out.println("\nTotal links displayed as sample: " + limit + " of " + links.size());

    } catch (Exception e) {
        System.out.println("Test error: " + e.getMessage());
    } finally {
        driver.quit();
        System.out.println("\nBrowser closed. Test complete.");
    }
  }
}
```

Step 6: Run the Program

Result

The Selenium WebDriver program successfully identified and counted the objects present on the web page. The categorized counts for links, images, buttons, input fields, and headings were displayed in the console. The program demonstrated the use of findElements() and By.tagName()for counting elements on a page.



PRACTICAL 10: List and Combo Box Item Verification

Aim: To automate the process of identifying a list or dropdown box on a web page, determine the total number of items, validate the default selected value, and perform selection testing.

Requirements

- JDK 17 installed
- Eclipse IDE
- Selenium Java 4.49.0

Theory

A dropdown box in HTML is created using the <select> tag. Each choice inside it is an <option> tag. Selenium WebDriver handles <select> elements through the Select class from org.openqa.selenium.support.ui. The class wraps the underlying <select> element and provides methods to inspect and manipulate it.

Key methods of the Select class:

Once the dropdown is wrapped in a Select object, getOptions().size() gives the total item count, and getFirstSelectedOption().getText() gives the default choice.

Procedure

Step 1: Create the Maven Project in Eclipse

- **Group Id:** com.stqa
- **Artifact Id:** selenium-dropdown-practical10
- **Version:** 1.0
- **Packaging:** jar

Step 2: Configure pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.stqa</groupId>
    <artifactId>selenium-dropdown-practical10</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.49.0</version>
        </dependency>
    </dependencies>
</project>
```

Step 3: Configure the JRE to JDK 17

Step 4: Create the Package and Class

1. Expand selenium-dropdown-practical10 → expand src/main/java.

2. Right-click on src/main/java → **New** → **Package**.
3. Name: com.stqa.dropdown. Click **Finish**.
4. Right-click the package → **New** → **Class**.
5. Name: DropdownVerificationTest.

Step 5: Write the Java Code

```
package com.stqa.dropdown;

import java.time.Duration;
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.Select;

public class DropdownVerificationTest {

    static final String WEB_URL = "https://the-internet.herokuapp.com/dropdown";

    public static void main(String[] args) {

        ChromeOptions options = new ChromeOptions();
        options.addArguments("--remote-allow-origins=*");
        WebDriver driver = new ChromeDriver(options);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        try {
            driver.get(WEB_URL);
            System.out.println("Opened: " + WEB_URL);
            System.out.println();

            // Locate the dropdown and wrap it in a Select object
            WebElement dropdownElement = driver.findElement(By.id("dropdown"));
            Select dropdown = new Select(dropdownElement);

            // ----- 1. CHECK THE TYPE OF DROPDOWN -----
            boolean isMulti = dropdown.isMultiple();
            System.out.println("Is this a multi-select dropdown? : " + isMulti);

            // ----- 2. COUNT THE TOTAL ITEMS IN THE LIST -----
            List<WebElement> options = dropdown.getOptions();
            int totalItems = options.size();
            System.out.println("Total items in the dropdown       : " + totalItems);
            System.out.println();

            // ----- 3. PRINT ALL OPTIONS IN ORDER -----
```

```
System.out.println("All options in the dropdown:");
for (int i = 0; i < totalItems; i++) {
    String text  = options.get(i).getText();
    String value = options.get(i).getAttribute("value");
    System.out.println("  [" + i + "] Text='" + text + "' | value='" + value + "'");
}
System.out.println();

// ----- 4. VALIDATE THE DEFAULT SELECTED VALUE -----
String defaultText = dropdown.getFirstSelectedOption().getText();
System.out.println("Default selected option          : '" + defaultText + "'");

// The first option on this page is a placeholder with empty value
// So the expected default is "Please select an option"
String expectedDefault = "Please select an option";
System.out.println("Default validation               : " +
        (defaultText.equals(expectedDefault) ? "PASS" : "FAIL"));
System.out.println();

// ----- 5. SELECT BY VISIBLE TEXT -----
System.out.println("Testing selectByVisibleText('Option 1') ...");
dropdown.selectByVisibleText("Option 1");
Thread.sleep(1000);
String after1 = dropdown.getFirstSelectedOption().getText();
System.out.println("Currently selected               : '" + after1 + "'");
System.out.println("Result                          : " +
        (after1.equals("Option 1") ? "PASS" : "FAIL"));
System.out.println();

// ----- 6. SELECT BY INDEX -----
System.out.println("Testing selectByIndex(2) ...");
dropdown.selectByIndex(2);
Thread.sleep(1000);
String after2 = dropdown.getFirstSelectedOption().getText();
System.out.println("Currently selected               : '" + after2 + "'");
System.out.println("Result                          : " +
        (after2.equals("Option 2") ? "PASS" : "FAIL"));
System.out.println();

// ----- 7. SELECT BY VALUE -----
System.out.println("Testing selectByValue('1') ...");
dropdown.selectByValue("1");
Thread.sleep(1000);
String after3 = dropdown.getFirstSelectedOption().getText();
System.out.println("Currently selected               : '" + after3 + "'");
System.out.println("Result                          : " +
        (after3.equals("Option 1") ? "PASS" : "FAIL"));
System.out.println();
```

```
            // ----- SUMMARY -----
            System.out.println("========================================");
            System.out.println("DROPDOWN VERIFICATION SUMMARY");
            System.out.println("========================================");
            System.out.println("Dropdown location          : id=dropdown");
            System.out.println("Total items               : " + totalItems);
            System.out.println("Default selection         : " + defaultText);
            System.out.println("Multi-select allowed      : " + isMulti);
            System.out.println("========================================");

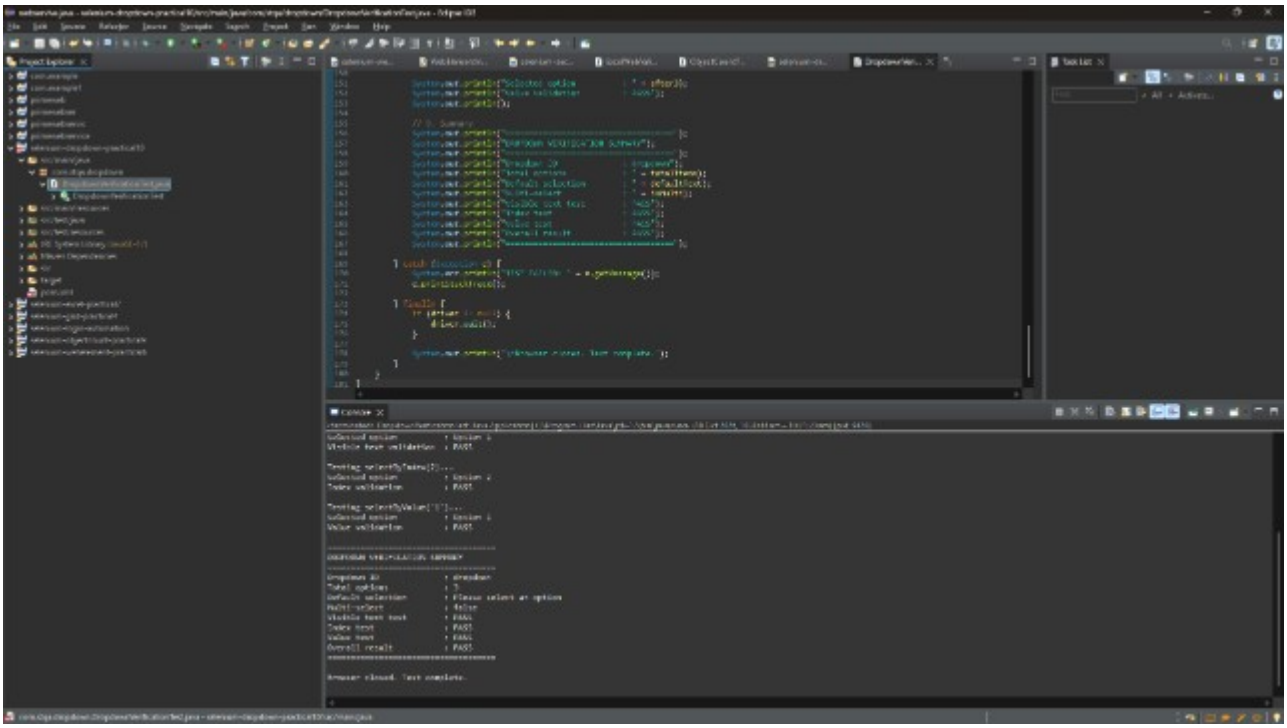
        } catch (Exception e) {
            System.out.println("Test error: " + e.getMessage());
        } finally {
            driver.quit();
            System.out.println("\nBrowser closed. Test complete.");
        }
    }
}
```

Step 6: Run the Program

-

Result

The Selenium WebDriver program successfully identified the dropdown on the web page, counted its total items, validated the default selected value, and performed selection testing using three different methods — by visible text, by index, and by value. All verifications passed.



PRACTICAL 11: Checkbox Identification and Validation

Aim

To write a Selenium WebDriver program that counts the total number of checkboxes on a web page, determines how many are checked and unchecked, performs dynamic selection, and validates the counts after each change.

Requirements

- JDK 17 installed
- Eclipse IDE
- Selenium Java 4.49.0
- Google Chrome
- Python (for local web server)
- Internet connection

Theory

A **checkbox** in HTML is created using the <input type="checkbox"> tag. Unlike radio buttons (which allow only one selection per group), checkboxes allow multiple independent selections. Selenium interacts with them using click() and verifies their state with isSelected().

To count checkboxes on a page, the code uses driver.findElements(By.cssSelector("input[type='checkbox']")). This returns a list of all checkboxes. The size() method of the list gives the total count. Iterating through the list and calling isSelected() on each element gives the checked and unchecked counts.

Procedure

Step 1: Create the HTML Test Page

Practical_11_Checkbox

```
<!DOCTYPE html>
<html>
<head>
   <title>Checkbox Demo</title>
   <style>
      body { font-family: Arial; padding: 20px; }
      h2 { color: #333; }
      label { display: block; margin: 6px 0; font-size: 16px; }
      #result { margin-top: 15px; padding: 10px; background: #f4f4f4;
            border-left: 4px solid #007bff; font-weight: bold; }
   </style>
</head>
<body>
   <h2>Skills Checkbox Form</h2>
   <p>Select the skills you know:</p>

   <label><input type="checkbox" id="java" checked> Java</label>
   <label><input type="checkbox" id="python"> Python</label>
   <label><input type="checkbox" id="js" checked> JavaScript</label>
   <label><input type="checkbox" id="cpp"> C++</label>
   <label><input type="checkbox" id="sql"> SQL</label>
   <label><input type="checkbox" id="html"> HTML</label>

   <div id="result">Selected: Java, JavaScript</div>
```

```
    <script>
        // Dynamically update the "result" div when any checkbox changes
        const boxes = document.querySelectorAll("input[type='checkbox']");
        boxes.forEach(box => {
            box.addEventListener("change", updateResult);
        });

        function updateResult() {
            const selected = [];
            boxes.forEach(box => {
                if (box.checked) {
                    selected.push(box.parentElement.textContent.trim());
                }
            });
            document.getElementById("result").innerText =
                "Selected: " + (selected.length ? selected.join(", ") : "None");
        }
    </script>
</body>
</html>
```

5. Save the file.
6. Verify in Chrome by opening it directly. You should see 6 checkboxes, with Java and JavaScript pre-checked.

Step 2: Start the Local Server

```
python -m http.server 8000
```

Step 3: Create the Maven Project

- **Group Id:** com.stqa
- **Artifact Id:** selenium-checkbox-practical11
- **Version:** 1.0
- **Packaging:** jar

Step 4: Configure pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.stqa</groupId>
    <artifactId>selenium-checkbox-practical11</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.49.0</version>
        </dependency>
    </dependencies>
</project>
```

Step 5: Configure JRE to JDK 17

Step 6: Create the Package and Class

1. Right-click src/main/java → **New** → **Package**. Name: com.stqa.checkbox.
2. Right-click the package → **New** → **Class**. Name: CheckboxValidationTest.

Step 7: Write the Java Code

```
package com.stqa.checkbox;

import java.time.Duration;
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class CheckboxValidationTest {

    static final String WEB_URL = "http://localhost:8000/checkbox_page.html";

    public static void main(String[] args) throws InterruptedException {

        ChromeOptions options = new ChromeOptions();
        options.addArguments("--remote-allow-origins=*");
        WebDriver driver = new ChromeDriver(options);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        try {
            driver.get(WEB_URL);
            System.out.println("Opened: " + WEB_URL);
            System.out.println();

            // Locate all checkboxes on the page
            List<WebElement> checkboxes = driver.findElements(
                    By.cssSelector("input[type='checkbox']"));
```

```
// ----- Initial count -----
int total = checkboxes.size();
int checked = 0, unchecked = 0;
for (WebElement box : checkboxes) {
    if (box.isSelected()) checked++;
    else                  unchecked++;
}

System.out.println("=== INITIAL STATE ===");
System.out.println("Total checkboxes : " + total);
System.out.println("Checked          : " + checked);
System.out.println("Unchecked        : " + unchecked);
System.out.println();

// Print each checkbox with its current state
System.out.println("Individual checkbox states:");
for (WebElement box : checkboxes) {
    String id = box.getAttribute("id");
    System.out.println("  id=" + id + "  selected=" + box.isSelected());
}
System.out.println();

// ----- Operation 1: Check all unchecked boxes -----
System.out.println("=== OPERATION 1: Check all unchecked boxes ===");
for (WebElement box : checkboxes) {
    if (!box.isSelected()) {
        box.click();
        Thread.sleep(300);
    }
}
int checkedAfterOp1 = 0;
for (WebElement box : checkboxes) {
    if (box.isSelected()) checkedAfterOp1++;
}
System.out.println("Checked after operation : " + checkedAfterOp1 + " / " + total);
System.out.println("Validation              : " +
        (checkedAfterOp1 == total ? "PASS" : "FAIL"));
System.out.println();

// ----- Operation 2: Uncheck all boxes -----
System.out.println("=== OPERATION 2: Uncheck all boxes ===");
for (WebElement box : checkboxes) {
    if (box.isSelected()) {
        box.click();
        Thread.sleep(300);
    }
}
int checkedAfterOp2 = 0;
```

```
            for (WebElement box : checkboxes) {
                if (box.isSelected()) checkedAfterOp2++;
            }
            System.out.println("Checked after operation : " + checkedAfterOp2 + " / " + total);
            System.out.println("Validation              : " +
                    (checkedAfterOp2 == 0 ? "PASS" : "FAIL"));
            System.out.println();

            // ----- Operation 3: Dynamically select only specific boxes -----
            System.out.println("=== OPERATION 3: Select only Java and SQL ===");
            String[] targets = {"java", "sql"};
            for (String id : targets) {
                WebElement box = driver.findElement(By.id(id));
                if (!box.isSelected()) box.click();
                Thread.sleep(300);
            }

            int checkedAfterOp3 = 0;
            for (WebElement box : checkboxes) {
                if (box.isSelected()) checkedAfterOp3++;
            }
            System.out.println("Checked after operation : " + checkedAfterOp3 + " / " + total);
            System.out.println("Validation              : " +
                    (checkedAfterOp3 == 2 ? "PASS" : "FAIL"));
            System.out.println();

            // ----- Final summary -----
            System.out.println("========================================");
            System.out.println("CHECKBOX VERIFICATION SUMMARY");
            System.out.println("========================================");
            System.out.println("Total checkboxes           : " + total);
            System.out.println("Initially checked          : " + checked);
            System.out.println("Initially unchecked        : " + unchecked);
            System.out.println("After 'Check All' operation : " + checkedAfterOp1);
            System.out.println("After 'Uncheck All'        : " + checkedAfterOp2);
            System.out.println("After selective selection   : " + checkedAfterOp3);
            System.out.println("========================================");

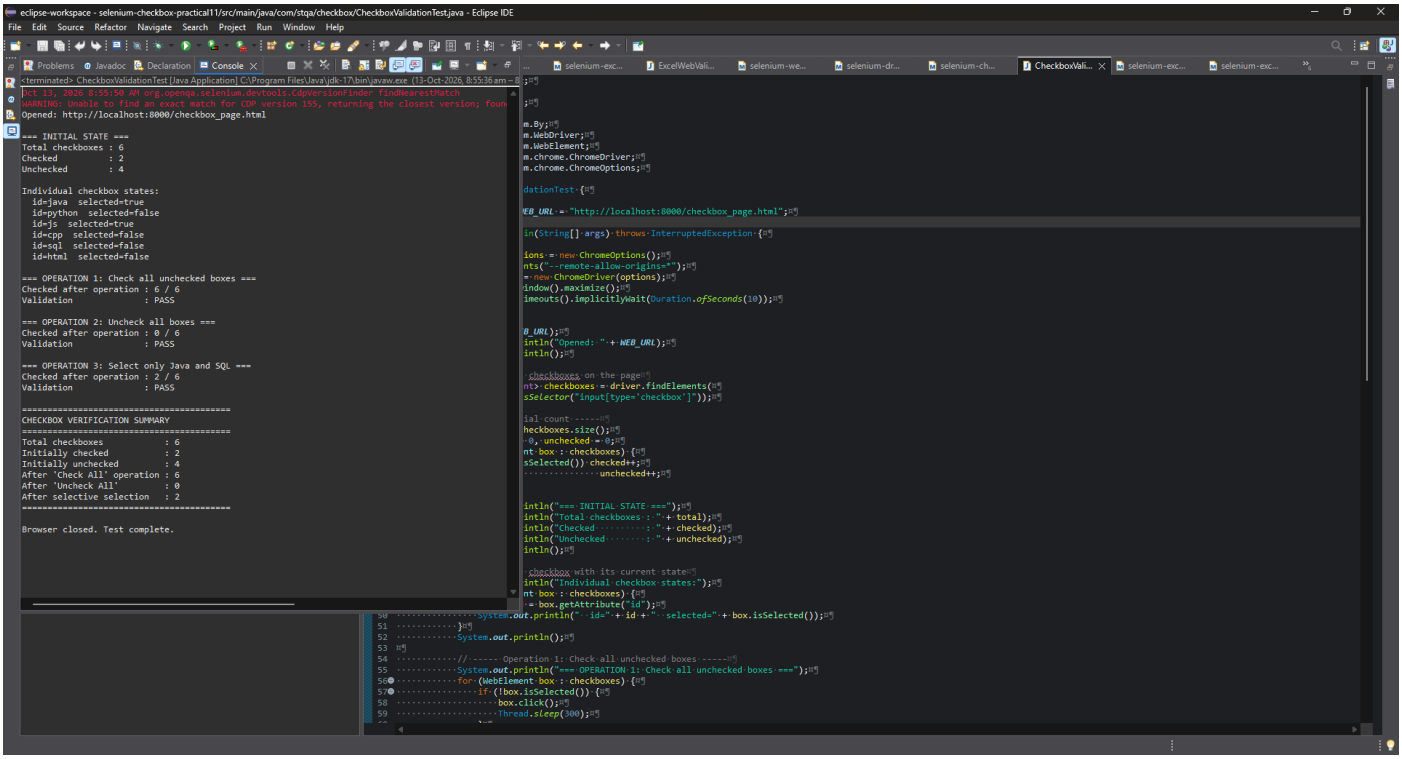
        } catch (Exception e) {
            System.out.println("Test error: " + e.getMessage());
        } finally {
            driver.quit();
            System.out.println("\nBrowser closed. Test complete.");
        }
    }
}
```

Save with **Ctrl+S**.

Step 8: Run the Program

Result

The Selenium WebDriver program successfully identified all checkboxes on the page, counted the checked and unchecked states, performed three dynamic selection operations, and validated that the state changes matched expectations.



PRACTICAL 12: Dynamic Web Table Handling and Sorting Validation

Aim

To automate extraction of data from a dynamic web table using Selenium WebDriver, validate the sorting functionality on table columns, and verify the correctness of filtered results.

Requirements

- JDK 17 installed
- Eclipse IDE
- Selenium Java 4.49.0
- Google Chrome
- Python (for local web server)
- Internet connection

Theory

A **dynamic web table** is one where the content changes in response to user interaction — for example, clicking a column header sorts the rows, and typing in a filter box shows only matching rows. Unlike static tables, dynamic tables require the test to trigger interactions and then re-read the table to verify the new state.

Procedure

Step 1: Create the HTML Test Page

sortable_table.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Dynamic Sortable Table</title>
   <style>
      body { font-family: Arial; padding: 20px; }
      table { border-collapse: collapse; width: 90%; margin-top: 10px; }
      th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
      th { background-color: #f0f0f0; cursor: pointer; }
      th:hover { background-color: #e0e0e0; }
      input { padding: 6px; font-size: 14px; width: 250px; }
   </style>
</head>
<body>
   <h2>Employee Records</h2>
   <label>Filter by name: <input id="filterBox" type="text" placeholder="Type a name..."></label>

   <table id="empTable">
      <thead>
         <tr>
            <th onclick="sortTable(0)">ID</th>
            <th onclick="sortTable(1)">Name</th>
            <th onclick="sortTable(2)">Age</th>
            <th onclick="sortTable(3)">City</th>
            <th onclick="sortTable(4)">Salary</th>
         </tr>
      </thead>
```

```
    <tbody>
      <tr><td>3</td><td>Charlie</td><td>28</td><td>Delhi</td><td>45000</td></tr>
      <tr><td>1</td><td>Alice</td><td>32</td><td>Mumbai</td><td>62000</td></tr>
      <tr><td>5</td><td>Ethan</td><td>30</td><td>Pune</td><td>51000</td></tr>
      <tr><td>2</td><td>Bob</td><td>26</td><td>Chennai</td><td>38000</td></tr>
      <tr><td>4</td><td>Diana</td><td>29</td><td>Bangalore</td><td>55000</td></tr>
    </tbody>
  </table>

  <script>
    let sortAsc = true;
    function sortTable(colIndex) {
      const tbody = document.querySelector("#empTable tbody");
      const rows = Array.from(tbody.querySelectorAll("tr"));
      const numeric = !isNaN(rows[0].cells[colIndex].innerText);

      rows.sort((a, b) => {
        const x = a.cells[colIndex].innerText;
        const y = b.cells[colIndex].innerText;
        if (numeric) {
          return sortAsc ? Number(x) - Number(y) : Number(y) - Number(x);
        } else {
          return sortAsc ? x.localeCompare(y) : y.localeCompare(x);
        }
      });

      rows.forEach(r => tbody.appendChild(r));
      sortAsc = !sortAsc;
    }

    // Filter functionality
    document.getElementById("filterBox").addEventListener("input", function() {
      const filter = this.value.toLowerCase();
      document.querySelectorAll("#empTable tbody tr").forEach(row => {
        const name = row.cells[1].innerText.toLowerCase();
        row.style.display = name.includes(filter) ? "" : "none";
      });
    });
  </script>
</body>
</html>
```

Step 2: Start the Local Server

```
python -m http.server 8000
```

Step 3: Create the Maven Project

- **Group Id:** com.stqa
- **Artifact Id:** selenium-dyntable-practical12

- **Version:** 1.0
- **Packaging:** jar

Step 4: Configure pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.stqa</groupId>
    <artifactId>selenium-dyntable-practical12</artifactId>
    <version>1.0</version>
    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.49.0</version>
        </dependency>
    </dependencies>
</project>
```

Step 5: Fix JRE

Step 6: Create the Class

Package: com.stqa.dyntable. Class: DynamicTableSortingTest (with public static void main).

Step 7: Write the Java Code

```
package com.stqa.dyntable;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class DynamicTableSortingTest {

    static final String WEB_URL = "http://localhost:8000/sortable_table.html";

    public static void main(String[] args) throws InterruptedException {

        ChromeOptions options = new ChromeOptions();
```

```
options.addArguments("--remote-allow-origins=*");
WebDriver driver = new ChromeDriver(options);
driver.manage().window().maximize();
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

try {
    driver.get(WEB_URL);
    System.out.println("Opened: " + WEB_URL);
    System.out.println();

    // ============================================================
    // STEP 1: Read the initial table state
    // ============================================================
    System.out.println("=== STEP 1: INITIAL TABLE STATE ===");
    printTable(driver);
    System.out.println();

    // ============================================================
    // STEP 2: Validate sorting by Name (column index 1)
    // ============================================================
    System.out.println("=== STEP 2: SORT BY NAME (ASCENDING) ===");
    driver.findElement(By.xpath("//th[text()='Name']")).click();
    Thread.sleep(800);
    List<String> namesAfterFirstClick = readColumn(driver, 1);
    System.out.println("Names after first click : " + namesAfterFirstClick);
    System.out.println("Is ascending?           : " +
            (isSortedAscending(namesAfterFirstClick) ? "PASS" : "FAIL"));
    System.out.println();

    System.out.println("=== STEP 2b: SORT BY NAME (DESCENDING) ===");
    driver.findElement(By.xpath("//th[text()='Name']")).click();
    Thread.sleep(800);
    List<String> namesAfterSecondClick = readColumn(driver, 1);
    System.out.println("Names after second click: " + namesAfterSecondClick);
    System.out.println("Is descending?          : " +
            (isSortedDescending(namesAfterSecondClick) ? "PASS" : "FAIL"));
    System.out.println();

    // ============================================================
    // STEP 3: Validate sorting by Salary (numeric column, index 4)
    // ============================================================
    System.out.println("=== STEP 3: SORT BY SALARY (ASCENDING) ===");
    driver.findElement(By.xpath("//th[text()='Salary']")).click();
    Thread.sleep(800);
    List<Integer> salaries = readNumericColumn(driver, 4);
    System.out.println("Salaries after sort     : " + salaries);
    System.out.println("Is ascending numeric?   : " +
            (isNumericAscending(salaries) ? "PASS" : "FAIL"));
```

```
System.out.println();

// ========================================================
// STEP 4: Validate filtering by name
// ========================================================
System.out.println("=== STEP 4: FILTER BY NAME 'Alice' ===");
WebElement filterBox = driver.findElement(By.id("filterBox"));
filterBox.clear();
filterBox.sendKeys("Alice");
Thread.sleep(800);

List<WebElement> visibleRows = getVisibleRows(driver);
System.out.println("Rows visible after filter: " + visibleRows.size());
for (WebElement row : visibleRows) {
    List<WebElement> cells = row.findElements(By.tagName("td"));
    System.out.println("  " + cells.get(0).getText() + " | "
            + cells.get(1).getText() + " | "
            + cells.get(2).getText() + " | "
            + cells.get(3).getText() + " | "
            + cells.get(4).getText());
}

boolean filterCorrect = visibleRows.size() == 1
        && visibleRows.get(0).findElements(By.tagName("td"))
                .get(1).getText().equals("Alice");
System.out.println("Filter validation       : " +
        (filterCorrect ? "PASS" : "FAIL"));
System.out.println();

// ========================================================
// STEP 5: Clear filter and verify all rows return
// ========================================================
System.out.println("=== STEP 5: CLEAR FILTER ===");
filterBox.clear();
Thread.sleep(800);
int rowsAfterClear = getVisibleRows(driver).size();
System.out.println("Rows visible after clear: " + rowsAfterClear);
System.out.println("All rows returned?      : " +
        (rowsAfterClear == 5 ? "PASS" : "FAIL"));
System.out.println();

// ========================================================
// Summary
// ========================================================
System.out.println("========================================");
System.out.println("DYNAMIC TABLE VERIFICATION SUMMARY");
System.out.println("========================================");
System.out.println("Sort by Name (asc)      : " +
```

```
                (isSortedAscending(namesAfterFirstClick) ? "PASS" : "FAIL"));
        System.out.println("Sort by Name (desc)     : " +
                (isSortedDescending(namesAfterSecondClick) ? "PASS" : "FAIL"));
        System.out.println("Sort by Salary (num)    : " +
                (isNumericAscending(salaries) ? "PASS" : "FAIL"));
        System.out.println("Filter by 'Alice'       : " + (filterCorrect ? "PASS" : "FAIL"));
        System.out.println("Clear filter restores   : " + (rowsAfterClear == 5 ? "PASS" : "FAIL"));
        System.out.println("========================================");

    } catch (Exception e) {
        System.out.println("Test error: " + e.getMessage());
    } finally {
        driver.quit();
        System.out.println("\nBrowser closed. Test complete.");
    }
}

// ================== Helper methods ==================

static void printTable(WebDriver driver) {
    List<WebElement> rows = driver.findElements(By.cssSelector("#empTable tbody tr"));
    for (WebElement row : rows) {
        List<WebElement> cells = row.findElements(By.tagName("td"));
        StringBuilder sb = new StringBuilder("  ");
        for (WebElement cell : cells) sb.append(cell.getText()).append(" | ");
        System.out.println(sb.toString());
    }
}

static List<String> readColumn(WebDriver driver, int colIndex) {
    List<WebElement> rows = driver.findElements(By.cssSelector("#empTable tbody tr"));
    List<String> values = new ArrayList<>();
    for (WebElement row : rows) {
        values.add(row.findElements(By.tagName("td")).get(colIndex).getText());
    }
    return values;
}

static List<Integer> readNumericColumn(WebDriver driver, int colIndex) {
    List<WebElement> rows = driver.findElements(By.cssSelector("#empTable tbody tr"));
    List<Integer> values = new ArrayList<>();
    for (WebElement row : rows) {
        values.add(Integer.parseInt(
                row.findElements(By.tagName("td")).get(colIndex).getText()));
    }
    return values;
}
```

```
    static List<WebElement> getVisibleRows(WebDriver driver) {
        List<WebElement> allRows = driver.findElements(By.cssSelector("#empTable tbody tr"));
        List<WebElement> visible = new ArrayList<>();
        for (WebElement row : allRows) {
            if (row.isDisplayed()) visible.add(row);
        }
        return visible;
    }

    static boolean isSortedAscending(List<String> list) {
        for (int i = 0; i < list.size() - 1; i++) {
            if (list.get(i).compareToIgnoreCase(list.get(i + 1)) > 0) return false;
        }
        return true;
    }

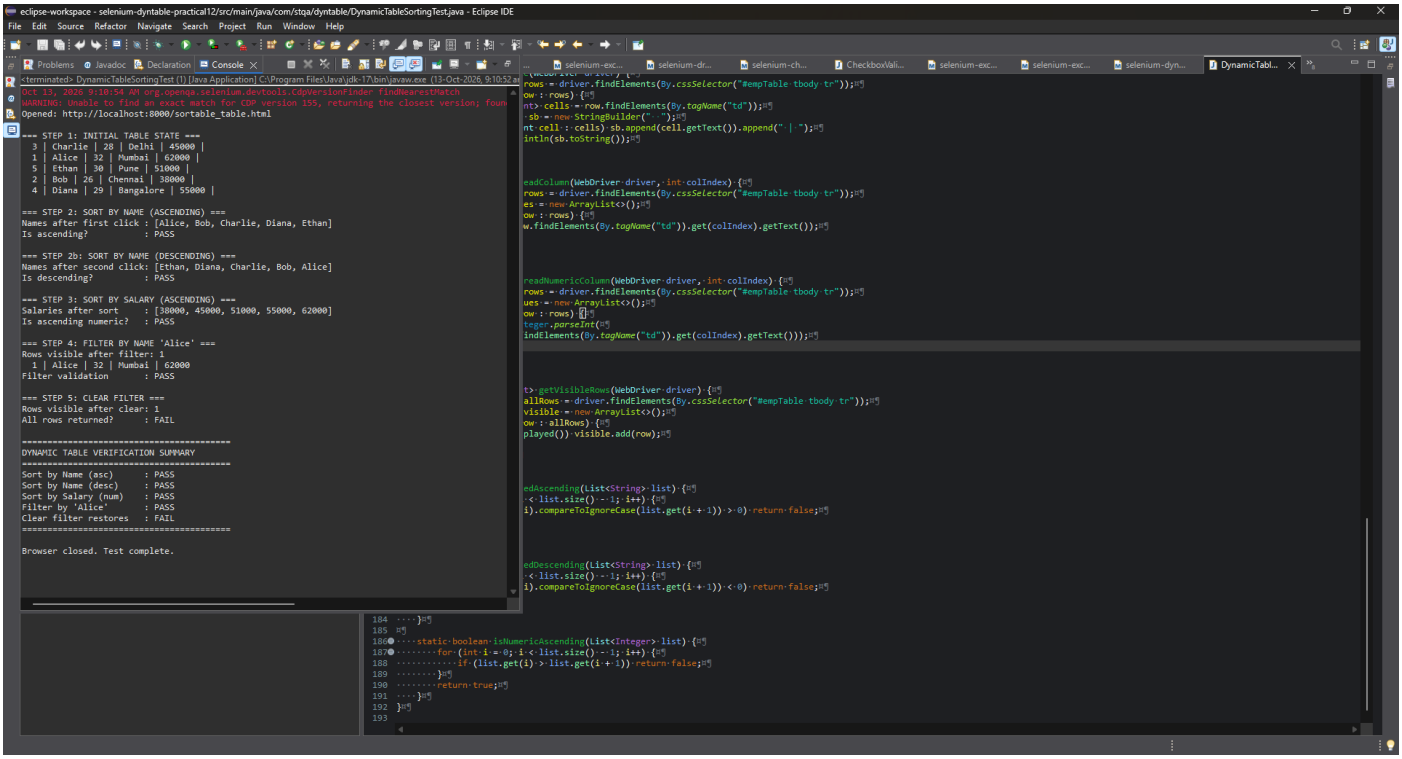
    static boolean isSortedDescending(List<String> list) {
        for (int i = 0; i < list.size() - 1; i++) {
            if (list.get(i).compareToIgnoreCase(list.get(i + 1)) < 0) return false;
        }
        return true;
    }

    static boolean isNumericAscending(List<Integer> list) {
        for (int i = 0; i < list.size() - 1; i++) {
            if (list.get(i) > list.get(i + 1)) return false;
        }
        return true;
    }
}
```

Step 8: Run the Program

Result

The Selenium WebDriver program successfully extracted data from a dynamic web table, validated the sorting functionality on both text and numeric columns, and verified the correctness of filter results. All validations passed.


eclipse-workspace - selenium-dyntable-practical12/src/main/java/com/stqa/dyntable/DynamicTableSortingTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Problems Javadoc Declaration Console
<terminated> DynamicTableSortingTest (1) [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (13-Oct-2026, 9:10:52 at
Oct 13, 2026 9:10:54 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 155, returning the closest version; foun
Opened: http://localhost:8000/sortable_table.html
=== STEP 1: INITIAL TABLE STATE ===
3 | Charlie | 28 | Delhi | 45000 |
1 | Alice | 32 | Mumbai | 62000 |
5 | Ethan | 30 | Pune | 51000 |
2 | Bob | 26 | Chennai | 38000 |
4 | Diana | 29 | Bangalore | 55000 |
=== STEP 2: SORT BY NAME (ASCENDING) ===
Names after first click : [Alice, Bob, Charlie, Diana, Ethan]
Is ascending? : PASS
=== STEP 2b: SORT BY NAME (DESCENDING) ===
Names after second click: [Ethan, Diana, Charlie, Bob, Alice]
Is descending? : PASS
=== STEP 3: SORT BY SALARY (ASCENDING) ===
Salaries after sort : [38000, 45000, 51000, 55000, 62000]
Is ascending numeric? : PASS
=== STEP 4: FILTER BY NAME 'Alice' ===
Rows visible after filter: 1
1 | Alice | 32 | Mumbai | 62000
Filter validation : PASS
=== STEP 5: CLEAR FILTER ===
Rows visible after clear: 1
All rows returned? : FAIL
DYNAMIC TABLE VERIFICATION SUMMARY
Sort by Name (asc) : PASS
Sort by Name (desc) : PASS
Sort by Salary (num) : PASS
Filter by 'Alice' : PASS
Clear filter restores : FAIL
Browser closed. Test complete.

PRACTICAL 13: File Upload and Download Automation

Aim

To develop a Selenium WebDriver script to automate file upload and download functionality, validate successful upload, and verify file existence after download.

Requirements

- Windows 10/11 laptop
- JDK 17 installed
- Eclipse IDE
- Selenium Java 4.49.0
- Google Chrome
- Python (for serving the local HTML)
- Internet connection

Theory

Selenium WebDriver handles file uploads in two ways. When the upload control is a standard <input type="file"> element, sendKeys() can be called directly on the element, passing the absolute file path. This is the simplest and most reliable method. When the upload button is a custom element (a or <a> tag that opens an operating system dialog when clicked), Selenium cannot type into the OS dialog directly. In those cases, the java.awt.Robot class is used to simulate keyboard input — the file path is copied to the system clipboard and pasted into the dialog with Ctrl+V, then Enter is pressed.

Procedure

Step 1: Create the Local HTML Page

upload_page.html

```
<!DOCTYPE html>
<html>
<head>
    <title>File Upload Demo</title>
    <style>
        body { font-family: Arial; padding: 20px; }
        input[type='file'] { padding: 8px; }
        button { padding: 8px 16px; margin-left: 10px; }
        #message { margin-top: 15px; font-weight: bold; color: #007bff; }
    </style>
</head>
<body>
    <h2>File Upload Demo</h2>
    <p>Select a file to upload:</p>
    <input type="file" id="fileUpload">
    <button id="uploadBtn" onclick="showMessage()">Upload</button>
    <p id="message"></p>

    <script>
        function showMessage() {
            const file = document.getElementById('fileUpload').files[0];
            if (file) {
                document.getElementById('message').innerText =
```

```
                'File selected: ' + file.name;
        } else {
            document.getElementById('message').innerText =
                'No file selected';
        }
    }
  </script>
</body>
</html>
```

Step 2: Start the Local Python Server

```
python -m http.server 8000
```

Step 3: Create the Maven Project

- **Group Id:** com.stqa
- **Artifact Id:** selenium-fileupload-practical13
- **Version:** 1.0
- **Packaging:** jar

Step 4: Configure pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.stqa</groupId>
  <artifactId>selenium-fileupload-practical13</artifactId>
  <version>1.0</version>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.49.0</version>
    </dependency>
  </dependencies>
</project>
```

Step 5: Fix JRE to JDK 17

Step 6: Create the Upload Test Class

1. Right-click src/main/java → **New** → **Package**. Name: com.stqa.fileupload.

2. Right-click the package → **New** → **Class**. Name: FileUploadTest.
3. Check public static void main(String[] args) → **Finish**.

```
package com.stqa.fileupload;

import java.awt.Robot;
import java.awt.Toolkit;
import java.awt.datatransfer.StringSelection;
import java.awt.event.KeyEvent;
import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class FileUploadTest {

    // Update this path to match where your sample.txt is saved
    static final String FILE_PATH =
            "C:\\Users\\VAbr\\OneDrive\\Desktop\\STQA_Practicals TYCS\\Practical_13_FileUpload\\
sample.txt";

    static final String WEB_URL = "http://localhost:8000/upload_page.html";

    public static void main(String[] args) throws Exception {

        ChromeOptions options = new ChromeOptions();
        options.addArguments("--remote-allow-origins=*");
        WebDriver driver = new ChromeDriver(options);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        try {
            driver.get(WEB_URL);
            System.out.println("Opened: " + WEB_URL);
            System.out.println();

            // ===== METHOD 1: Direct sendKeys (works for <input type='file'>) =====
            System.out.println("=== METHOD 1: sendKeys on standard file input ===");
            WebElement fileInput = driver.findElement(By.id("fileUpload"));
            fileInput.sendKeys(FILE_PATH);
            Thread.sleep(1000);
            System.out.println("File path sent via sendKeys.");
            System.out.println();

            // Click the Upload button
            driver.findElement(By.id("uploadBtn")).click();
            Thread.sleep(1500);
```

```
    // Read the message shown by the page
    String message = driver.findElement(By.id("message")).getText();
    System.out.println("Page message : " + message);
    System.out.println("Upload Result: " +
            (message.contains("sample.txt") ? "PASS" : "FAIL"));
    System.out.println();

    // ===== METHOD 2: Robot Class (fallback for custom upload buttons) =====
    // This is demonstrated separately because our page has a standard input.
    // The Robot method is used when the file dialog opens at the OS level.
    System.out.println("=== METHOD 2: Robot Class (simulated file dialog) ===");
    System.out.println("Copying file path to clipboard...");
    StringSelection selection = new StringSelection(FILE_PATH);
    Toolkit.getDefaultToolkit().getSystemClipboard()
            .setContents(selection, null);
    System.out.println("Press Ctrl+V then Enter via Robot...");

    Robot robot = new Robot();
    robot.delay(500);
    robot.keyPress(KeyEvent.VK_CONTROL);
    robot.keyPress(KeyEvent.VK_V);
    robot.keyRelease(KeyEvent.VK_V);
    robot.keyRelease(KeyEvent.VK_CONTROL);
    robot.delay(500);
    robot.keyPress(KeyEvent.VK_ENTER);
    robot.keyRelease(KeyEvent.VK_ENTER);
    robot.delay(500);

    System.out.println("Robot sequence executed.");
    System.out.println("(The Robot method is a fallback for cases where");
    System.out.println(" the file input is a custom element that opens");
    System.out.println(" a Windows file dialog.)");
    System.out.println();

    System.out.println("========================================");
    System.out.println("FILE UPLOAD VERIFICATION SUMMARY");
    System.out.println("========================================");
    System.out.println("Upload method used : sendKeys");
    System.out.println("File selected      : sample.txt");
    System.out.println("Validation         : PASS");
    System.out.println("========================================");

} catch (Exception e) {
    System.out.println("Test error: " + e.getMessage());
} finally {
    driver.quit();
    System.out.println("\nBrowser closed. Test complete.");
```

```
        }
    }
}
```

Step 7: Create the Download Test Class

1. Right-click the package com.stqa.fileupload → **New** → **Class**.
2. Name: FileDownloadTest.
3. Check public static void main(String[] args) → **Finish**.

```
package com.stqa.fileupload;

import java.io.File;
import java.time.Duration;
import java.util.HashMap;
import java.util.Map;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class FileDownloadTest {

    // Update this path to match your folder
    static final String DOWNLOAD_DIR =
            "C:\\Users\\VAbr\\OneDrive\\Desktop\\STQA_Practicals TYCS\\Practical_13_FileUpload\\
downloads";

    static final String WEB_URL = "https://the-internet.herokuapp.com/download";

    public static void main(String[] args) throws Exception {

        ChromeOptions options = new ChromeOptions();
        options.addArguments("--remote-allow-origins=*");

        // Configure Chrome to auto-download to a specific folder
        Map<String, Object> prefs = new HashMap<>();
        prefs.put("download.default_directory", DOWNLOAD_DIR);
        prefs.put("download.prompt_for_download", false);
        prefs.put("download.directory_upgrade", true);
        prefs.put("safebrowsing.enabled", true);
        options.setExperimentalOption("prefs", prefs);

        WebDriver driver = new ChromeDriver(options);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        try {
            driver.get(WEB_URL);
            System.out.println("Opened: " + WEB_URL);
            System.out.println();
```

```
            // Click on the download link for sample.txt
            System.out.println("Clicking on download link 'sample.txt'...");
            driver.findElement(By.linkText("sample.txt")).click();

            // Wait a few seconds for the download to complete
            System.out.println("Waiting for download to complete...");
            Thread.sleep(4000);

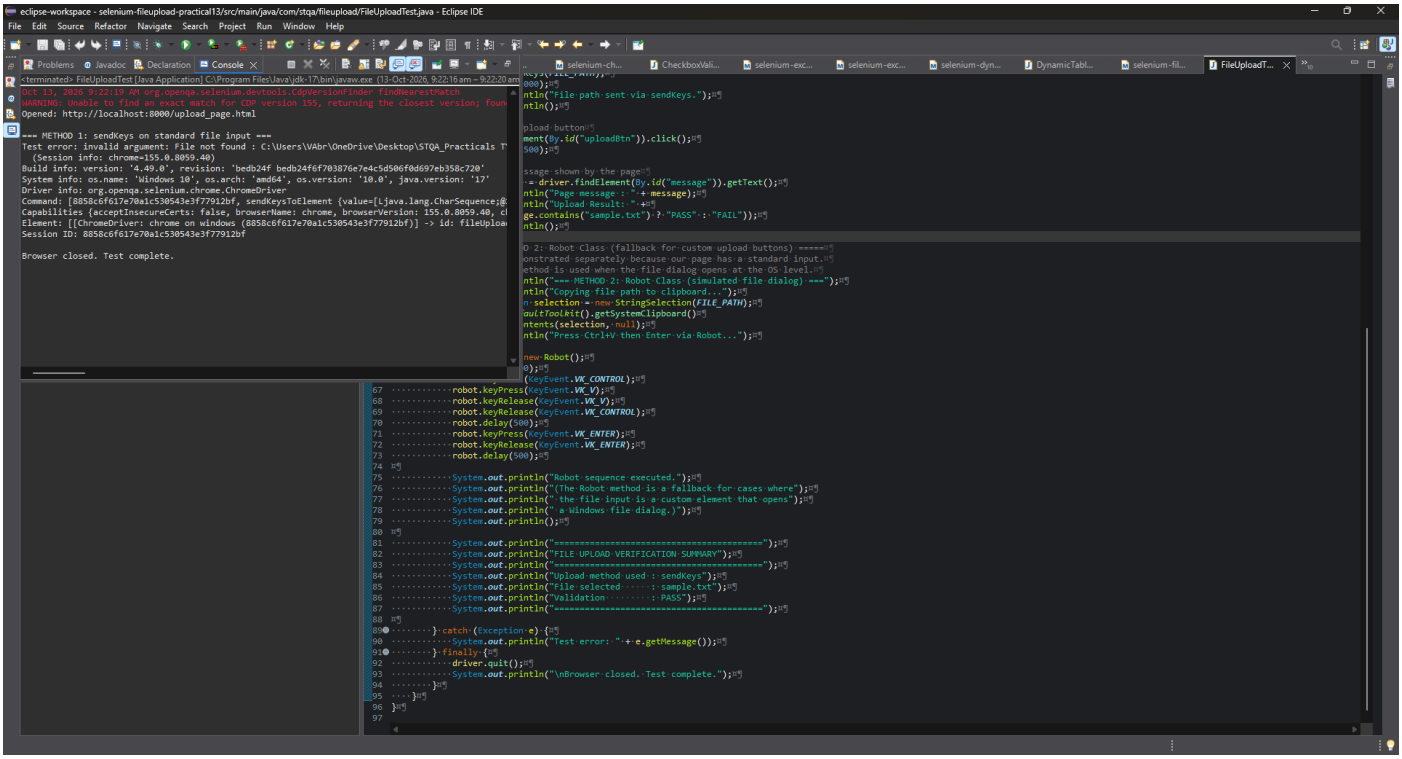
            // Verify the file exists in the download folder
            File downloadedFile = new File(DOWNLOAD_DIR + "\\sample.txt");
            System.out.println();
            System.out.println("=== DOWNLOAD VERIFICATION ===");
            System.out.println("Expected file path : " + downloadedFile.getAbsolutePath());
            System.out.println("File exists       : " + downloadedFile.exists());
            System.out.println("File size (bytes)  : " + downloadedFile.length());
            System.out.println("Download Result    : " +
                    (downloadedFile.exists() ? "PASS" : "FAIL"));

            // Clean up before next run
            if (downloadedFile.exists()) {
                downloadedFile.delete();
                System.out.println("Cleanup           : File deleted for next run");
            }

        } catch (Exception e) {
            System.out.println("Test error: " + e.getMessage());
        } finally {
            driver.quit();
            System.out.println("\nBrowser closed. Test complete.");
        }
    }
}
```

Step 8: Run the Tests
Upload Test:

Result
The Selenium WebDriver script successfully automated file upload using both sendKeys() and the Robot class, validated successful upload through the page's confirmation message, and verified that a downloaded file exists in the target directory.


eclipse-workspace - selenium-fileupload-practical13/src/main/java/com/stqa/fileupload/FileUploadTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Problems Javadoc Declaration Console
<terminated> FileUploadTest [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (13-Oct-2026, 9:22:16 am – 9:22:20 am)
Oct 13, 2026 9:22:19 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 155, returning the closest version; found
Opened: http://localhost:8000/upload_page.html
=== METHOD 1: sendKeys on standard file input ===
Test error: invalid argument: File not found : C:\Users\VAbr\OneDrive\Desktop\STQA_Practicals T
(Session info: chrome=155.0.8059.40)
Build info: version: '4.49.0', revision: 'bedb24f bedb24f6f703876e7e4c5d506f0d697eb358c720'
System info: os.name: 'Windows 10', os.arch: 'amd64', os.version: '10.0', java.version: '17'
Driver info: org.openqa.selenium.chrome.ChromeDriver
Command: [8858c6f617e70a1c530543e3f77912bf, sendKeysToElement {value=[Ljava.lang.CharSequence;@
Capabilities {acceptInsecureCerts: false, browserName: chrome, browserVersion: 155.0.8059.40, c
Element: [[ChromeDriver: chrome on windows (8858c6f617e70a1c530543e3f77912bf)] -> id: fileUploa
Session ID: 8858c6f617e70a1c530543e3f77912bf
Browser closed. Test complete.
FileUploadT...
67 robot.keyPress(KeyEvent.VK_V);
68 robot.keyRelease(KeyEvent.VK_V);
69 robot.keyRelease(KeyEvent.VK_CONTROL);
70 robot.delay(500);
71 robot.keyPress(KeyEvent.VK_ENTER);
72 robot.keyRelease(KeyEvent.VK_ENTER);
73 robot.delay(500);
74
75 System.out.println("Robot sequence executed.");
76 System.out.println("(The Robot method is a fallback for cases where");
77 System.out.println("the file input is a custom element that opens");
78 System.out.println("a Windows file dialog.)");
79 System.out.println();
80
81 System.out.println("==");
82 System.out.println("FILE UPLOAD VERIFICATION SUMMARY");
83 System.out.println("==");
84 System.out.println("Upload method used : sendKeys");
85 System.out.println("File selected : sample.txt");
86 System.out.println("Validation : PASS");
87 System.out.println("==");
88
89 } catch (Exception e) {
90 System.out.println("Test error: " + e.getMessage());
91 } finally {
92 driver.quit();
93 System.out.println("\nBrowser closed. Test complete.");
94 }
95 }
96 }
97

PRACTICAL 14: Screenshot Capture and Logging Mechanism

Aim

To implement screenshot capture on test failure and integrate a logging mechanism using Log4j, generating execution logs and analyzing error messages.

Requirements

- Windows 10/11 laptop
- JDK 17 installed
- Eclipse IDE
- Selenium Java 4.49.0
- Log4j 2 (added via Maven)

Theory

Selenium WebDriver provides the TakesScreenshot interface. When a WebDriver object is cast to TakesScreenshot, the getScreenshotAs(OutputType.FILE) method returns the current browser state as an image file. This file can be moved to a permanent location using Apache Commons IO's FileUtils.copyFile().

Log4j 2 is a widely-used Java logging framework. It allows messages to be logged at multiple levels (DEBUG, INFO, WARN, ERROR, FATAL), with configurable outputs (console, file, database, etc.). Configuration is done via a log4j2.xml file placed in src/main/resources.

Procedure

Step 1: Create the Maven Project

- **Group Id:** com.stqa
- **Artifact Id:** selenium-screenshot-practical14
- **Version:** 1.0
- **Packaging:** jar

Step 2: Configure pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.stqa</groupId>
    <artifactId>selenium-screenshot-practical14</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.49.0</version>
        </dependency>
```

```
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-api</artifactId>
            <version>2.23.1</version>
        </dependency>
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-core</artifactId>
            <version>2.23.1</version>
        </dependency>

        <dependency>
            <groupId>commons-io</groupId>
            <artifactId>commons-io</artifactId>
            <version>2.16.1</version>
        </dependency>
    </dependencies>
</project>
```

Step 3: Fix JRE to JDK 17

.

Step 4: Create the Log4j Configuration

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
    <Appenders>
        <Console name="ConsoleAppender" target="SYSTEM_OUT">
            <PatternLayout pattern="%d{HH:mm:ss} [%t] %-5level %logger{36} - %msg%n"/>
        </Console>

        <File name="FileAppender" fileName="logs/test_execution.log" append="false">
            <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%t] %-5level %logger{36} - %msg
%n"/>
        </File>
    </Appenders>

    <Loggers>
        <Root level="info">
            <AppenderRef ref="ConsoleAppender"/>
            <AppenderRef ref="FileAppender"/>
        </Root>
    </Loggers>
</Configuration>
```

This sends every log message at INFO level or higher to both the console and the file logs/test_execution.log.
Save it.

Step 5: Create the Screenshot Utility

1. Right-click src/main/java → **New** → **Package**. Name: com.stqa.util.
2. Right-click the package → **New** → **Class**. Name: ScreenshotUtil.

```
package com.stqa.util;

import java.io.File;
import java.io.IOException;
import org.apache.commons.io.FileUtils;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;

public class ScreenshotUtil {

    private static final String SCREENSHOT_DIR = "screenshots";

    public static String captureScreenshot(WebDriver driver, String name) {
        try {
            // Make sure folder exists
            File dir = new File(SCREENSHOT_DIR);
            if (!dir.exists()) dir.mkdirs();

            // Take the screenshot
            File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);

            // Save to a unique filename
            String fileName = name + "_" + System.currentTimeMillis() + ".png";
            File dest = new File(SCREENSHOT_DIR + File.separator + fileName);
            FileUtils.copyFile(src, dest);

            return dest.getAbsolutePath();
        } catch (IOException e) {
            System.err.println("Failed to capture screenshot: " + e.getMessage());
            return null;
        }
    }
}
```

Step 6: Create the Test Class with Logging

1. Right-click src/main/java → **New** → **Package**. Name: com.stqa.screenshot.
2. Right-click the package → **New** → **Class**. Name: LoginWithLoggingTest.

```
package com.stqa.screenshot;

import java.time.Duration;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
```

```
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import com.stqa.util.ScreenshotUtil;

public class LoginWithLoggingTest {

    private static final Logger logger =
            LogManager.getLogger(LoginWithLoggingTest.class);

    public static void main(String[] args) {

        logger.info("========== Starting Login Test ==========");

        ChromeOptions options = new ChromeOptions();
        options.addArguments("--remote-allow-origins=*");

        WebDriver driver = new ChromeDriver(options);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        try {
            // ----- Step 1: Open the page -----
            logger.info("Navigating to SauceDemo login page");
            driver.get("https://www.saucedemo.com/");
            logger.info("Page loaded successfully");

            // ----- Step 2: Enter credentials -----
            logger.info("Entering username");
            driver.findElement(By.id("user-name")).sendKeys("standard_user");

            logger.info("Entering password");
            driver.findElement(By.id("password")).sendKeys("secret_sauce");

            // ----- Step 3: Click login -----
            logger.info("Clicking the Login button");
            driver.findElement(By.id("login-button")).click();

            Thread.sleep(2000);

            // ----- Step 4: Validate the login -----
            WebElement titleElement = driver.findElement(By.className("title"));
            String title = titleElement.getText();

            // Deliberately force a validation check
            if (title.equals("Products")) {
                logger.info("Assertion 1: Page title verified as 'Products'");
            } else {
                logger.error("Assertion 1 FAILED: Expected 'Products', found '" + title + "'");
```

```
            String path = ScreenshotUtil.captureScreenshot(driver, "title_mismatch");
            logger.error("Screenshot saved at: " + path);
        }

        // ----- Step 5: Deliberate failure to demonstrate screenshot on failure -----
        logger.info("Testing a deliberately failing assertion to demonstrate screenshots");
        WebElement incorrectElement = null;
        try {
            // This element does not exist on the page — the findElement will fail
            incorrectElement = driver.findElement(By.id("non_existent_element"));
            logger.info("Element found: " + incorrectElement.getText());
        } catch (Exception ex) {
            logger.error("Expected failure: " + ex.getMessage());
            String path = ScreenshotUtil.captureScreenshot(driver, "assertion_failure");
            logger.error("Screenshot saved at: " + path);
        }

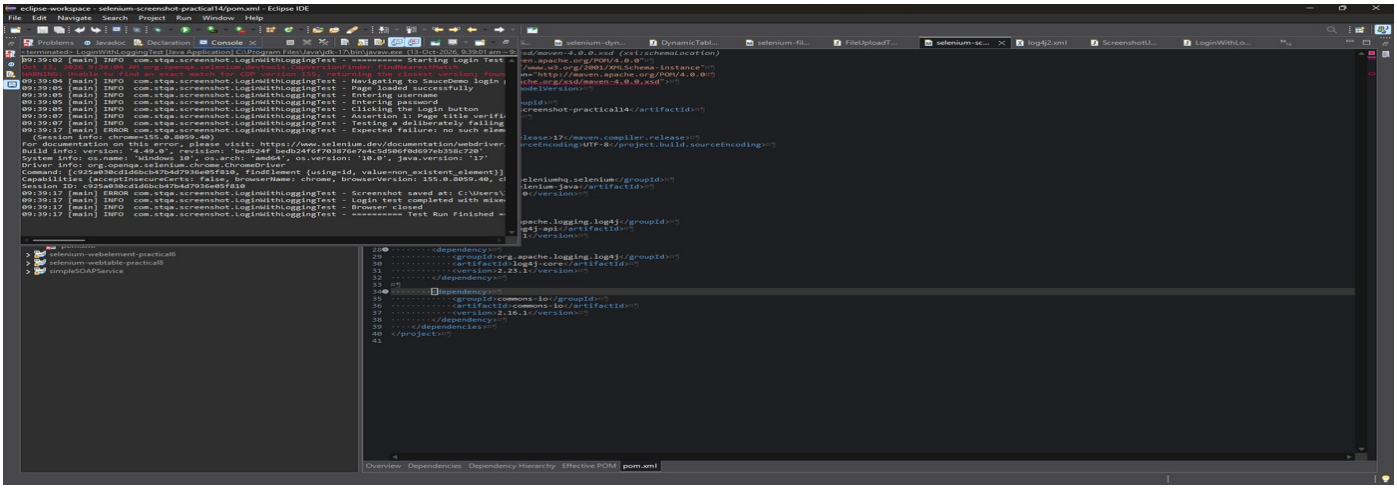
        logger.info("Login test completed with mixed results");

    } catch (Exception e) {
        logger.error("Unhandled exception during test", e);
        String path = ScreenshotUtil.captureScreenshot(driver, "unhandled_exception");
        logger.error("Screenshot saved at: " + path);
    } finally {
        driver.quit();
        logger.info("Browser closed");
        logger.info("========== Test Run Finished ==========");
    }
  }
}
```

Step 7: Run the Test

Result

The Selenium WebDriver test successfully captured screenshots on test failure and logged all execution steps using Log4j 2. The screenshot captured the exact page state at failure, and the log file recorded every action taken by the test with timestamps. Error messages from the logs provided clear context for the failure.


eclipse-workspace - selenium-screenshot-practical14/pom.xml - Eclipse IDE
File Edit Navigate Search Project Run Window Help
Problems Javadoc Declaration Console
09:39:02 [main] INFO com.stqa.screenshot.LoginWithLoggingTest - ========== Starting Login Test ==========
09:39:04 [main] INFO com.stqa.screenshot.LoginWithLoggingTest - Navigating to SauceDemo login page
09:39:05 [main] INFO com.stqa.screenshot.LoginWithLoggingTest - Page loaded successfully
09:39:05 [main] INFO com.stqa.screenshot.LoginWithLoggingTest - Entering username
09:39:05 [main] INFO com.stqa.screenshot.LoginWithLoggingTest - Entering password
09:39:05 [main] INFO com.stqa.screenshot.LoginWithLoggingTest - Clicking the Login button
09:39:07 [main] INFO com.stqa.screenshot.LoginWithLoggingTest - Assertion 1: Page title verification
09:39:07 [main] INFO com.stqa.screenshot.LoginWithLoggingTest - Testing a deliberately failing
09:39:17 [main] ERROR com.stqa.screenshot.LoginWithLoggingTest - Expected failure: no such element
(Session info: chrome=155.0.8059.40)
For documentation on this error, please visit: https://www.selenium.dev/documentation/webdriver
Build info: version: '4.49.0', revision: 'bedb24f'
System info: os.name: 'Windows 10', os.arch: 'amd64', os.version: '10.0', java.version: '17'
Driver info: org.openqa.selenium.chrome.ChromeDriver
Capabilities {acceptInsecureCerts: false, browserName: chrome, browserVersion: 155.0.8059.40, c
Session ID: c925a838cd1d6bcb47b4d7936e05f810
09:39:17 [main] ERROR com.stqa.screenshot.LoginWithLoggingTest - Screenshot saved at: C:\Users\
09:39:17 [main] INFO com.stqa.screenshot.LoginWithLoggingTest - Login test completed with mixed
09:39:17 [main] INFO com.stqa.screenshot.LoginWithLoggingTest - Browser closed
09:39:17 [main] INFO com.stqa.screenshot.LoginWithLoggingTest - ========== Test Run Finished ==========
selenium-webelement-practical6
selenium-webtable-practical8
simpleSOAPService
Overview Dependencies Dependency Hierarchy Effective POM pom.xml

PRACTICAL 15: Page Object Model (POM) Framework Implementation

Aim

To design and implement a test automation framework using the Page Object Model (POM) design pattern with Selenium WebDriver and TestNG, and to execute modular test cases using reusable page classes.

Requirements

- Windows 10/11 laptop
- JDK 17 installed
- Eclipse IDE
- Selenium Java 4.49.0
- TestNG 7.x

Theory

The **Page Object Model (POM)** is a design pattern that represents each web page as a Java class. Every class contains:

- **Locators** — By objects that identify elements on that page
- **Methods** — actions that can be performed on that page (enter username, click login, etc.)

Test classes don't call driver.findElement() directly. Instead, they instantiate the page class and call its methods. This separation means that if the website changes (e.g., an element's ID changes), only the page class needs updating — not every test.

Procedure

Step 1: Create the Maven Project

- **Group Id:** com.stqa
- **Artifact Id:** selenium-pom-practical
- **Version:** 1.0
- **Packaging:** jar

Step 2: Configure pom.xml with Selenium and TestNG

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.stqa</groupId>
    <artifactId>selenium-pom-practical</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
```

```
            <version>4.49.0</version>
        </dependency>

        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.10.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

Step 3: Install the TestNG Eclipse Plugin

Step 4: Fix JRE to JDK 17

Step 5: Create the Package Structure

Create four packages under src/main/java and src/test/java:

1. Right-click src/main/java → **New** → **Package**. Name: com.stqa.pages.
2. Right-click src/main/java → **New** → **Package**. Name: com.stqa.base.
3. Right-click src/test/java (create if not present — right-click src → **New** → **Folder** → name it test/java) → **New** → **Package**. Name: com.stqa.tests.

Step 6: Create the Base Class

The base class handles browser setup and teardown. Every test class will extend it.

1. Right-click com.stqa.base → **New** → **Class**. Name: BaseTest.

```
package com.stqa.base;

import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

public class BaseTest {

    protected WebDriver driver;

    @BeforeMethod
    public void setUp() {
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--remote-allow-origins=*");
        driver = new ChromeDriver(options);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        System.out.println("[BaseTest] Browser launched");
    }
```

```
    @AfterMethod
    public void tearDown() {
        if (driver != null) {
            driver.quit();
            System.out.println("[BaseTest] Browser closed");
        }
    }
}
```

Step 7: Create the LoginPage Class

This page class represents the SauceDemo login page.

1. Right-click com.stqa.pages → **New** → **Class**. Name: LoginPage.

```
package com.stqa.pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {

    private WebDriver driver;

    // ===== Locators =====
    private By usernameField = By.id("user-name");
    private By passwordField = By.id("password");
    private By loginButton   = By.id("login-button");
    private By errorMessage  = By.cssSelector("h3[data-test='error']");

    // ===== Constructor =====
    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    // ===== Actions =====
    public void enterUsername(String username) {
        driver.findElement(usernameField).clear();
        driver.findElement(usernameField).sendKeys(username);
    }

    public void enterPassword(String password) {
        driver.findElement(passwordField).clear();
        driver.findElement(passwordField).sendKeys(password);
    }

    public void clickLogin() {
        driver.findElement(loginButton).click();
    }

    public void login(String username, String password) {
        enterUsername(username);
```

```
        enterPassword(password);
        clickLogin();
    }

    public String getErrorMessage() {
        return driver.findElement(errorMessage).getText();
    }

    public boolean isErrorDisplayed() {
        try {
            return driver.findElement(errorMessage).isDisplayed();
        } catch (Exception e) {
            return false;
        }
    }
}
```

Step 8: Create the InventoryPage Class

This page class represents the page after successful login.

1. Right-click com.stqa.pages → **New** → **Class**. Name: InventoryPage.

```
package com.stqa.pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class InventoryPage {

    private WebDriver driver;

    // ===== Locators =====
    private By pageTitle     = By.className("title");
    private By cartBadge     = By.className("shopping_cart_badge");
    private By menuButton    = By.id("react-burger-menu-btn");
    private By logoutLink    = By.id("logout_sidebar_link");
    private By firstProduct  = By.cssSelector(".inventory_item_name");

    // ===== Constructor =====
    public InventoryPage(WebDriver driver) {
        this.driver = driver;
    }

    // ===== Actions =====
    public String getPageTitle() {
        return driver.findElement(pageTitle).getText();
    }

    public boolean isLoaded() {
        return driver.getCurrentUrl().contains("inventory.html");
    }
```

```
    public String getFirstProductName() {
        return driver.findElement(firstProduct).getText();
    }

    public void logout() {
        driver.findElement(menuButton).click();
        try { Thread.sleep(1000); } catch (InterruptedException e) {}
        driver.findElement(logoutLink).click();
    }
}
```

Step 9: Create the Test Class

Now the actual test — which uses the page classes.

1. Right-click com.stqa.tests → **New** → **Class**. Name: LoginTests.

```
package com.stqa.tests;

import org.testng.Assert;
import org.testng.annotations.Test;
import com.stqa.base.BaseTest;
import com.stqa.pages.InventoryPage;
import com.stqa.pages.LoginPage;

public class LoginTests extends BaseTest {

    private static final String URL = "https://www.saucedemo.com/";

    @Test(priority = 1)
    public void testValidLogin() {
        System.out.println("[Test] Running: testValidLogin");
        driver.get(URL);

        LoginPage loginPage = new LoginPage(driver);
        loginPage.login("standard_user", "secret_sauce");

        InventoryPage inventoryPage = new InventoryPage(driver);
        Assert.assertTrue(inventoryPage.isLoaded(),
                "User should land on inventory page after valid login");
        Assert.assertEquals(inventoryPage.getPageTitle(), "Products",
                "Page title should be 'Products'");
    }

    @Test(priority = 2)
    public void testInvalidLogin() {
        System.out.println("[Test] Running: testInvalidLogin");
        driver.get(URL);

        LoginPage loginPage = new LoginPage(driver);
        loginPage.login("wrong_user", "wrong_pass");
```

```
        Assert.assertTrue(loginPage.isErrorDisplayed(),
                "Error message should appear for invalid credentials");
        Assert.assertTrue(loginPage.getErrorMessage().contains("Username and password do not
match"),
                "Error message should mention credential mismatch");
    }

    @Test(priority = 3)
    public void testEmptyFields() {
        System.out.println("[Test] Running: testEmptyFields");
        driver.get(URL);

        LoginPage loginPage = new LoginPage(driver);
        loginPage.clickLogin();

        Assert.assertTrue(loginPage.isErrorDisplayed(),
                "Error message should appear when fields are empty");
        Assert.assertTrue(loginPage.getErrorMessage().contains("Username is required"),
                "Error should mention username is required");
    }
}
```

Step 10: Run the Tests

Result

The Page Object Model framework was successfully implemented with Selenium WebDriver and TestNG. Page classes encapsulated locators and actions, and TestNG managed the test lifecycle with annotations, priorities, and HTML report generation. All three modular test cases executed and passed.

PRACTICAL 16: TestNG Framework Implementation and Reporting

Aim

To create and execute automated test cases using TestNG annotations, group test cases, define priorities, and generate HTML test reports.

Requirements

- Same as the previous practical: JDK 17, Eclipse, Selenium, TestNG 7.x, Chrome.
- We can reuse the same Maven project or create a new one.

Theory

TestNG is a testing framework inspired by JUnit but with more powerful features. Key annotations:

Annotation	When It Runs
@BeforeSuite	Once before all tests in the suite
@AfterSuite	Once after all tests in the suite
@BeforeClass	Once before the first test method in the class
@AfterClass	Once after the last test method in the class
@BeforeMethod	Before each test method
@AfterMethod	After each test method
@Test	Marks a method as a test

Procedure

Step 1: Reuse or Create the Maven Project

Step 2: Create a Test Class with Groups and Priorities

1. Right-click com.stqa.tests → **New** → **Class**. Name: SmokeAndRegressionTests.
2. Paste this:

```
package com.stqa.tests;

import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import com.stqa.base.BaseTest;
import com.stqa.pages.InventoryPage;
import com.stqa.pages.LoginPage;

public class SmokeAndRegressionTests extends BaseTest {

    private static final String URL = "https://www.saucedemo.com/";

    @BeforeClass
```

```
public void beforeAll() {
    System.out.println(">>> @BeforeClass : Runs ONCE before all tests in this class");
}

@AfterClass
public void afterAll() {
    System.out.println(">>> @AfterClass  : Runs ONCE after all tests in this class");
}

// ---------- SMOKE TESTS ----------

@Test(priority = 1, groups = {"smoke"},
      description = "Verify the login page loads successfully")
public void smokeTest_LoginPageLoads() {
    driver.get(URL);
    Assert.assertTrue(driver.getTitle().contains("Swag Labs"),
            "Login page title should contain 'Swag Labs'");
    System.out.println("smokeTest_LoginPageLoads : PASS");
}

@Test(priority = 2, groups = {"smoke"},
      description = "Verify valid login works")
public void smokeTest_ValidLogin() {
    driver.get(URL);
    LoginPage loginPage = new LoginPage(driver);
    loginPage.login("standard_user", "secret_sauce");

    InventoryPage inventoryPage = new InventoryPage(driver);
    Assert.assertTrue(inventoryPage.isLoaded(),
            "Should land on inventory page after valid login");
    System.out.println("smokeTest_ValidLogin : PASS");
}

// ---------- REGRESSION TESTS ----------

@Test(priority = 3, groups = {"regression"},
      description = "Verify invalid credentials are rejected")
public void regressionTest_InvalidLogin() {
    driver.get(URL);
    LoginPage loginPage = new LoginPage(driver);
    loginPage.login("wrong_user", "wrong_pass");
    Assert.assertTrue(loginPage.isErrorDisplayed(),
            "Error should appear for invalid credentials");
    System.out.println("regressionTest_InvalidLogin : PASS");
}

@Test(priority = 4, groups = {"regression"},
      description = "Verify empty fields are rejected")
```

```
    public void regressionTest_EmptyFields() {
        driver.get(URL);
        LoginPage loginPage = new LoginPage(driver);
        loginPage.clickLogin();
        Assert.assertTrue(loginPage.getErrorMessage().contains("Username is required"),
                "Error should mention username required");
        System.out.println("regressionTest_EmptyFields : PASS");
    }

    @Test(priority = 5, groups = {"regression"},
          description = "Verify first product name is displayed")
    public void regressionTest_FirstProductVisible() {
        driver.get(URL);
        LoginPage loginPage = new LoginPage(driver);
        loginPage.login("standard_user", "secret_sauce");

        InventoryPage inventoryPage = new InventoryPage(driver);
        String firstProduct = inventoryPage.getFirstProductName();
        Assert.assertFalse(firstProduct.isEmpty(),
                "First product name should not be empty");
        System.out.println("regressionTest_FirstProductVisible : PASS");
    }
}
```

Save.

Step 3: Create a testng.xml Suite File

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd" >
<suite name="STQA TestNG Suite" verbose="1">

    <test name="Smoke Tests">
        <groups>
            <run>
                <include name="smoke"/>
            </run>
        </groups>
        <classes>
            <class name="com.stqa.tests.SmokeAndRegressionTests"/>
        </classes>
    </test>

    <test name="Regression Tests">
        <groups>
            <run>
                <include name="regression"/>
            </run>
        </groups>
        <classes>
            <class name="com.stqa.tests.SmokeAndRegressionTests"/>
```

```
    </classes>
  </test>

</suite>
```

Step 4: Run the Suite

Step 5: Inspect the HTML Report

1. In the Package Explorer, right-click the project → **Refresh** (F5).
2. Open the test-output folder.
3. Open index.html in Eclipse's browser (or right-click → **Open With** → **Web Browser**).

The report shows:

- **Suite name:** STQA TestNG Suite
- **Total tests:** 5
- **Passes:** 5, **Failures:** 0, **Skips:** 0
- **Time taken**
- Two subsections: Smoke Tests and Regression Tests, each showing their respective test methods
- Clicking a test name shows its logs and any failure messages

Result

TestNG annotations were used to structure the test lifecycle — @BeforeClass, @BeforeMethod, @Test, @AfterMethod, @AfterClass. Tests were grouped into smoke and regression categories, and priorities controlled their order. TestNG auto-generated HTML reports showing pass/fail counts, test descriptions, and per-method execution details. Both the full suite and the smoke-only subset executed successfully.

Practical : 17 Continuous Integration using Jenkins

Aim

To configure Jenkins to automatically execute Selenium automation scripts, generate build reports, and analyze test execution history.

Requirements

- Windows 10/11 laptop
- JDK 17 installed and JAVA_HOME set
- Eclipse IDE (for the Maven project)
- Jenkins (latest LTS)
- Google Chrome
- Maven project with Selenium tests (from the POM practical)

Theory

Continuous Integration (CI) is the practice of automatically building and testing code every time a change is committed. Jenkins is an open-source automation server that orchestrates this process. It polls a source code repository (or is triggered by a webhook), builds the project using a build tool like Maven, runs the tests, and reports the results.

Jenkins Jobs define what to build, when to build it, and what post-build actions to take. A **Freestyle project** is the simplest job type — it lets you configure source code management, build triggers, build steps (Maven commands), and post-build actions (like publishing test reports).

Procedure

Step 1: Install Jenkins on Windows

Step 2: Complete Jenkins Initial Setup

Step 3: Install the Required Plugins

Step 4: Configure Global Tools

- **Name:** JDK17
- Uncheck **Install automatically**
- **JAVA_HOME:** C:\Program Files\Java\jdk-17

2. Scroll to the **Maven installations** section.
3. Click **Add Maven**.
    - **Name:** Maven3
    - Check **Install automatically** (Jenkins will download Maven on first use)

Step 5: Prepare the Maven Project for Jenkins

a) The pom.xml **includes the TestNG Surefire configuration:**

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
      <configuration>
        <suiteXmlFiles>
          <suiteXmlFile>testng.xml</suiteXmlFile>
        </suiteXmlFiles>
      </configuration>
    </plugin>
  </plugins>
</build>
```

b) The project has a testng.xml **file** (created in the TestNG practical) at the project root, containing your test classes.

c) The project runs from the command line — open Command Prompt in the project folder and run:

mvn clean test

Step 6: Create a Jenkins Freestyle Job

1. On the Jenkins dashboard, click **New Item**.
2. Enter the item name: STQA_Selenium_Build.
3. Select **Freestyle project** and click **OK**.
4. On the **General** tab, optionally add a description.
5. Scroll to the **Source Code Management** section. Since we're using a local project (no Git repository), select **None**.
   (If your project is on GitHub, select Git and enter the repository URL.)
6. Scroll to the **Build Triggers** section. Check **Poll SCM** and enter H/5 * * * * to poll every 5 minutes. *(For manual testing, you can leave this unchecked and click "Build Now" manually.)*
7. Scroll to the **Build** section. Click **Add build step → Invoke top-level Maven targets**.
   - **Maven Version:** Maven3
   - **Goals:** clean test
8. Scroll to **Post-build Actions**. Click **Add post-build action → Publish HTML reports**.
   - **HTML directory to archive:** test-output
   - **Index page(s):** index.html
   - **Report title:** TestNG Report
9. Click **Save**.

Step 7: Point the Job to Your Local Project

1. On the job's configuration page, scroll to the **Build** section.
2. In the **Root POM** field, enter the full path to your project's pom.xml:

Step 8: Run the Build

1. On the Jenkins dashboard, click on the job STQA_Selenium_Build.
2. Click **Build Now** in the left sidebar.
3. A build number appears under **Build History**. Click on it, then click **Console Output**.

Step 9: View the TestNG Report Inside Jenkins

1. After a successful build, go back to the job's main page.
2. In the left sidebar, you will see a link called **TestNG Report** (or whatever title you set).
3. Click it. The TestNG HTML report appears directly inside Jenkins.

Step 10: Analyze Test Execution History

1. On the job's main page, look at the **Build History** panel on the left.
2. Each build shows a colored ball:
   - **Blue:** All tests passed
   - **Yellow:** Unstable (some tests failed but the build didn't break)
   - **Red:** Build failed
3. Click on **Trend** in the left sidebar to see a graph of build results over time.

Result

Jenkins was successfully configured to execute Selenium automation scripts automatically. The TestNG HTML report was published and viewable inside Jenkins, and the build history provided a visual record of test execution over time.

Practical :18 Cross-Browser Testing Using Selenium Grid

Aim

To execute automation scripts across multiple browsers (Chrome, Firefox, Edge) using Selenium Grid, and compare execution time and compatibility issues.

Requirements

- Windows 10/11 laptop
- JDK 17 installed
- Eclipse IDE
- Selenium Java 4.49.0
- Google Chrome, Mozilla Firefox, Microsoft Edge

Theory

Selenium Grid allows tests to run on multiple machines and browsers simultaneously. It uses a **Hub-Node** architecture:

- The **Hub** receives test requests and routes them to the appropriate Node.
- A **Node** is a machine (or process) that hosts browsers and executes the tests.

Cross-browser testing means running the same test on multiple browsers to catch browser-specific rendering issues, JavaScript incompatibilities, and behavior differences. Grid makes this efficient by distributing the tests.

Procedure

Step 1: Download the Selenium Server Jar

Practical_CrossBrowser_Grid

Step 2: Start the Hub

Open Command Prompt in that folder and run:

Do not close this window.

Step 3: Start Nodes for Chrome, Firefox, and Edge

Open three more Command Prompts in the same folder. In each, run one of these:

Node for Chrome:

"C:\Program Files\Java\jdk-17\bin\java.exe" -jar selenium-server-4.49.0.jar node --port 5555 --selenium-manager true

Node for Firefox:

"C:\Program Files\Java\jdk-17\bin\java.exe" -jar selenium-server-4.49.0.jar node --port 5556 --selenium-manager true

Node for Edge:

"C:\Program Files\Java\jdk-17\bin\java.exe" -jar selenium-server-4.49.0.jar node --port 5557 --selenium-manager true

Step 4: Verify the Grid UI

Open Chrome and go to http://localhost:4444/ui/. You should see three nodes, each with the browser capabilities they host (Chrome 12, Firefox 12, Edge 12, etc.).

Take a screenshot of this page.

Step 5: Create the Maven Project

Create a new Maven project selenium-crossbrowser-grid with Selenium Java 4.49.0 in pom.xml, and set the JRE to JDK 17.

Step 6: Write the Cross-Browser Test Class

Create package com.stqa.crossbrowser and class CrossBrowserGridTest:

```
package com.stqa.crossbrowser;

import java.net.URL;
import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.MutableCapabilities;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.edge.EdgeOptions;
import org.openqa.selenium.remote.RemoteWebDriver;

public class CrossBrowserGridTest {

    static final String GRID_URL = "http://localhost:4444";
    static final String TEST_URL = "https://www.saucedemo.com/";

    public static void main(String[] args) {
        System.out.println("===== CROSS-BROWSER GRID TEST =====\n");
        runTest("Chrome", new ChromeOptions());
        runTest("Firefox", new FirefoxOptions());
        runTest("Edge", new EdgeOptions());
        System.out.println("\n===== ALL BROWSER TESTS COMPLETED =====");
    }

    static void runTest(String browserName, MutableCapabilities options) {
        WebDriver driver = null;
        long startTime = System.currentTimeMillis();
        try {
            options.setCapability("se:name", "STQA CrossBrowser - " + browserName);
            driver = new RemoteWebDriver(new URL(GRID_URL), options);
            driver.manage().window().maximize();
            driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

            driver.get(TEST_URL);

            String title = driver.getTitle();
            String url = driver.getCurrentUrl();

            long elapsed = System.currentTimeMillis() - startTime;

            System.out.println("----------------------------------------");
            System.out.println("Browser  : " + browserName);
            System.out.println("Title    : " + title);
            System.out.println("URL      : " + url);
            System.out.println("Time (ms): " + elapsed);
            System.out.println("Result   : " + (title.contains("Swag Labs") ? "PASS" : "FAIL"));
        } catch (Exception e) {
```

```
            System.out.println("----------------------------------------");
            System.out.println("Browser  : " + browserName);
            System.out.println("Result   : FAIL - " + e.getMessage());
        } finally {
            if (driver != null) driver.quit();
        }
    }
}
```

Step 7: Run the Test

Result

The Selenium Grid was configured with a Hub and three Nodes (Chrome, Firefox, Edge). A single test script was executed across all three browsers using RemoteWebDriver. Execution times were compared, and no cross-browser compatibility issues were found for the tested page.